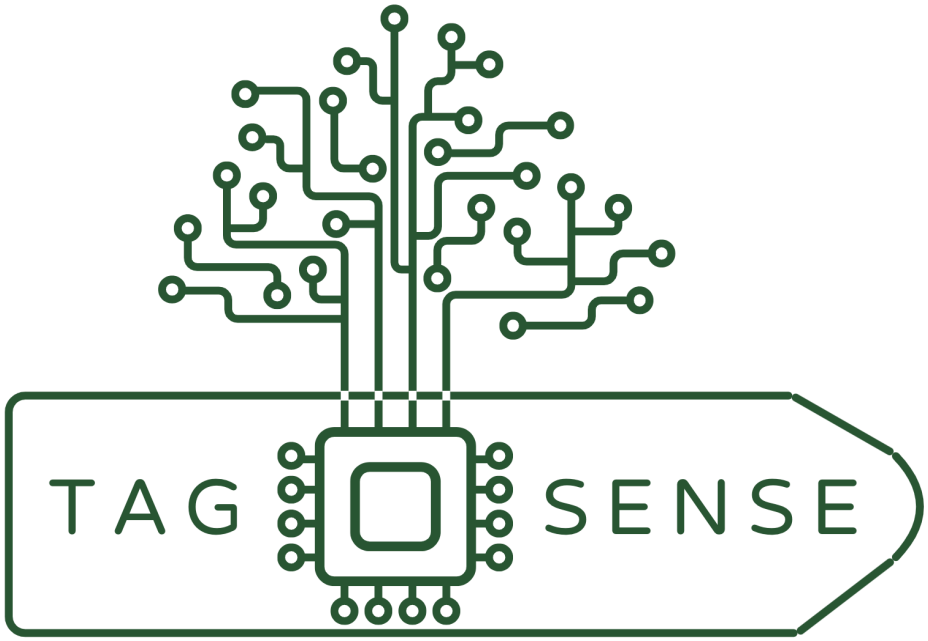


ENGINEER 4O16A Final Report: Reusable Smart Clothing Tag

Group 20

Brian Ho - 400254939 - hob14
Ben Miles - 400307241 - milesb3
Danielle Fong - 400333388 - fongd5
Isaac Kong - 400322178 - kongi2
Justin Covach - 400243271 - covachj

Table of Contents

Introduction

As of 2024's Q3, the United States has spent a total of USD 433.87 billion on clothing, with an increase in apparel spending from 2021 to 2024 of 15% [1]. This development signifies a growing clothing market as household income data grows by around 0.2% yearly; one can pinpoint that clothing consumerism is becoming an expanding market [1]. As the economy rebounds from a post-pandemic era, a magnification of the need for athletic and sports apparel can also be identified. In Canada, the sports apparel industry has seen an average annual growth of 25.5% in total revenue [2].

Thus, with a flourishing market with an abundance of consumer spending, a closer lens will reveal that retail stores require some bolstering and improvements. According to Zendesk's 2022 Customer Experience Report, 77% of shoppers were unsatisfied with the length of lines and wait times, and 73% of customers would abandon an interaction if they had to wait more than 5 minutes [2]. The long wait times to purchase clothing harm a consumer's experience and may negatively impact a company's sales. A retail store's priority is the customer experience, and long wait times are a common problem, especially during peak shopping seasons.

A contributing factor to lengthy queue times is the inefficiency in the usage of barcode scanning and security tag removal. These processes can be simplified to decrease the average shopper's wait time in lines. The downsides of barcode scanning are potential misreads, the need for clear lines, and slow input due to system issues.

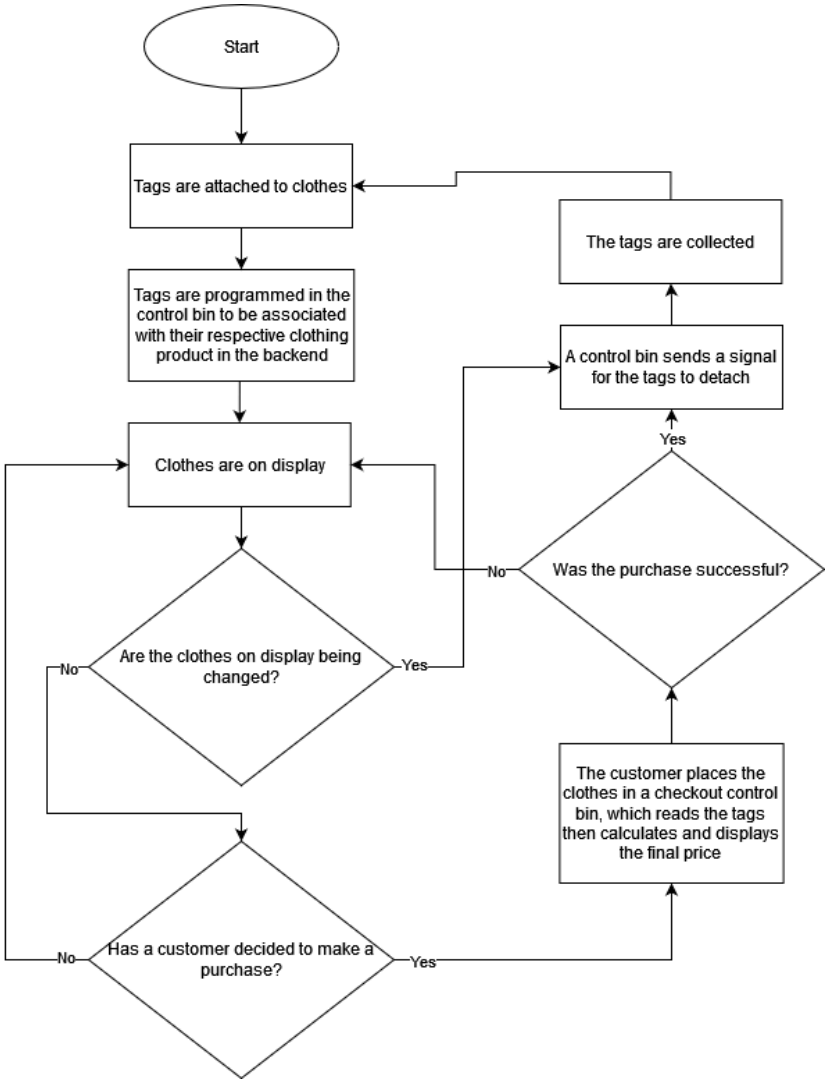
Finally, hang tags contribute to textile waste, as many are discarded post-transaction, emphasizing the reliance on landfills in North America. When defining hang tags on clothing, they are typically composed of two elements: the tag and the string or attachment that fastens it to the garment. While there is a wide variety of materials used in both tags and strings, the most common is a paper tag and a nylon string to secure the tag to clothing [3]. The paper tag can typically be around 3" x 1.5" and the nylon string around 2" [3]. While the paper tag can be recycled, the nylon string is non-degradable and non-recyclable. In North America, the average number of annual sales of individual garments is 10 billion units [3]. When considering the 2" length of the tags, this results in an estimated total of 20 billion inches of hang tag waste, equivalent to approximately 12.6 times the circumference of the Earth. The global environmental impact of this waste is substantial and poses a serious threat to our ecosystem.

Our goal is to increase retail self-checkout efficiency, lower retail waste, and improve the overall shopping experience. The RFID system aims to minimize wait times by cutting down on lines and replacing traditional physical tags with reusable devices to lower a business's carbon footprint. In addition, the new method will decrease the company's labour costs and automate aspects of the shopping process.

Project Goals and Planned Approach

This project consists of a clothing tag system for retail stores that will include a checkout system and smart tag. The checkout system will connect to a database of items and will be able to communicate with the smart tags. The smart tags themselves will be identifiable via RFID tags, reusable, reprogrammable, and released automatically from purchased clothes upon receipt of a signal from the checkout system.

For the initial setup of the system, the button on the smart tag is manually pressed to lock in a clothing item between the clothing pin. During checkout, the clothing item with the smart tag attached is placed into the control bin. The RFID tag in the smart tag will be detected and appear on the checkout screen. The PCB will receive a signal to perform the unlocking mechanism when the items are confirmed for purchase. During this time, the battery-powered PCB will activate the actuator, pulling the locking rod to release the clothing pin from the clothing item. Once the purchase is complete, the clothing item is ready to be taken, and the smart tag is prepared to be reused. The figure below outlines the system workflow of the methodology of how the product will work.

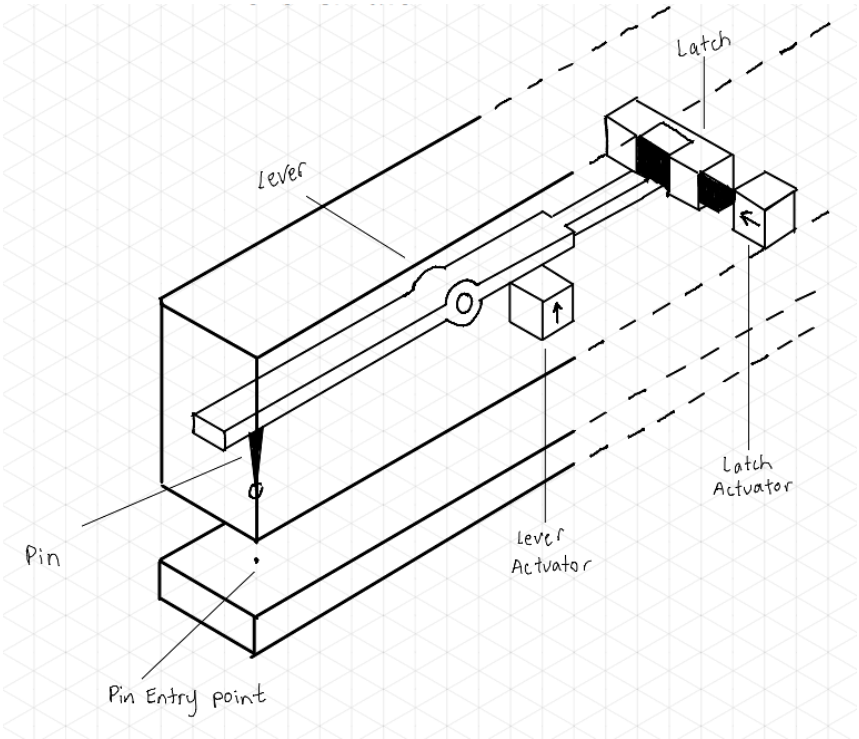


Our initial project design consists of a 3D printed reusable smart clothing tag, a control bin, and a back-end product database.

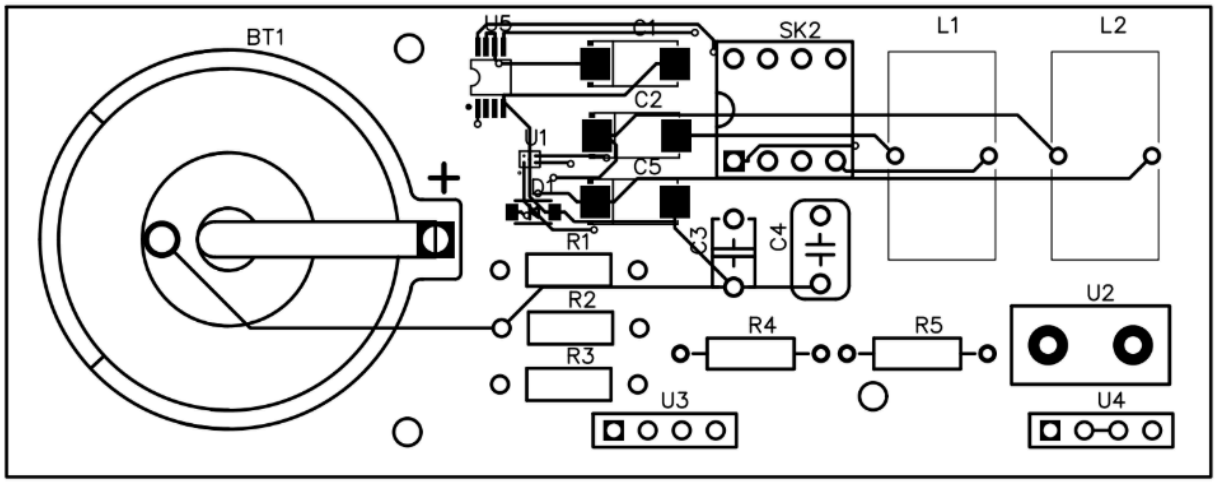
Reusable Smart Tag

The reusable smart tag is a device we intend to replace current paper or plastic tags used in clothing stores. The tag will be composed of an RFID circuit, an actuator, a PCB, and a 3D-printed case. The purpose of the RFID circuit is to hold a unique identifier to link the tag to the product it represents. The actuator will be used to attach the tag to the clothing product it represents. The PCB will be used to control the actuator, either opening or closing it, based on commands received through RF signals from the control bin. Lastly, the 3D-printed case will hold all the components of the tag together, and protect them from damage due to incidents such as falls. The tag should be the same size in area as current tags, though it may be larger in depth to accommodate the actuator and PCB. It should also consume very little energy while attached to closed and be very durable such that it can be reused many times.

Initial Locking Actuator Design:
The initial concept we decided to pursue further was a piezoelectric actuator as it seemed to fit the requirements we needed for this specific device. One of the main driving factors is the durability of the piezoelectric actuators and their ability to have a long working lifespan. Since the tag is a highly repetitive device, it is essential that it can last a long time for re-usability purposes. Another aspect about them that looked promising is their response time. This will help improve the user experience. This will help improve the user experience.



Initial PCB Design:
Utilization of a coin battery and a boost converter to power all required loads, creating a rechargeable tag with a portable and compact element to design.



Control Bin

The control bin will be used when programming the tags and at customer checkout. To accomplish this, it will require RFID scanners, a circuit to communicate with the reusable smart tags' PCBs, and a CPU for processing this data. It will also be connected to a monitor to interface with the user of the system.

Backend and Interface

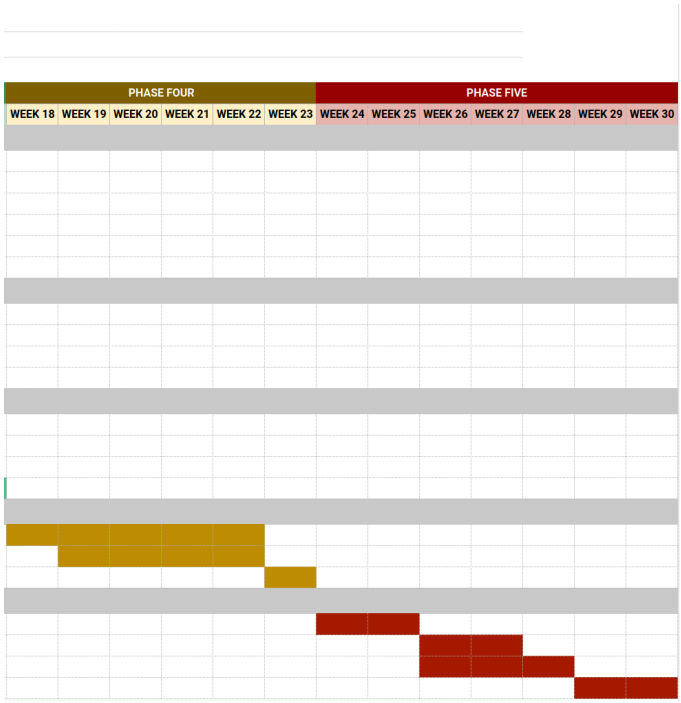
The data backend and interface will be an application designed to associate a tag's RFID with its product information and then demonstrate the pertinent information to the user of the system. This will require a database of products. Each product will have a price, description, and associated RFID number. Before displaying the clothes to customers, the stores should be able to use the interface (connected to the control bin) to make this product-RFID association. At checkout, this association will be removed once the customer has paid.

Scheduling

Below is a sample Gantt chart of our progress.

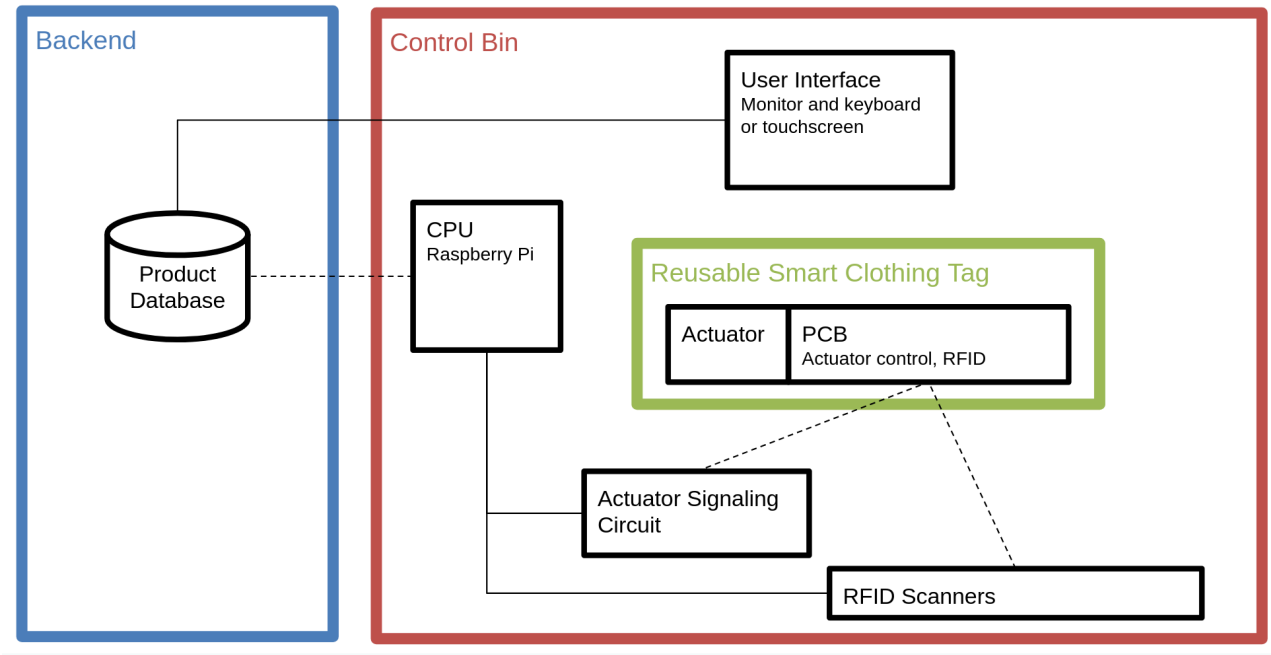
PROJECT TITLE	Reusable Smart Clothing Tag
PROJECT MEMBERS	Benjamin Miles, Brian Ho, Danielle Fong, Isaac Kong, Justin Covach
COURSE	ENGINEER 4016A/B
DATE	3/25/25

WBS NUMBER	TASK TITLE	START DATE	DUE DATE	DURATION	PCT OF TASK COMPLETE
1	**Initial Planning and Proposal**				
1.1	Brainstorm Ideas	9/9/24	9/12/24	3	100%
1.2	Research	9/12/24	9/30/24	18	100%
1.3	Finalize Initial Idea	9/24/24	10/2/24	8	100%
1.4	Present Idea and Get Approval	10/2/24	10/4/24	2	100%
1.5	Prepare Proposal	10/4/24	10/21/24	17	100%
1.6	Submit Proposal and Presentation	10/21/24	10/29/24	8	100%
2	**Preparing Project Build**				
2.1	Create BOM	11/4/24	11/6/24	2	100%
2.2	Design PCB	11/4/24	11/15/24	11	95%
2.2	Quote Materials Required	11/4/24	11/8/24	4	80%
2.3	Finalize Materials and Order Parts	11/11/24	11/20/24	9	80%
3	**Software Build**				
3.1	Implement actuator control program	11/25/24	12/6/24	11	100%
3.2	Implement RFID scanner program	12/6/24	12/20/24	14	100%
3.3	Implement system backend database	12/20/24	12/27/24	7	100%
3.4	Create Control Bin user interfaces	12/27/24	1/10/25	13	10%
4	**Hardware Build**				
4.1	Creating Smart Tag	1/6/25	2/7/25	31	100%
4.2	Creating Checkout System	1/13/25	2/7/25	24	90%
4.3	Testing Hardware	2/10/25	2/14/25	4	90%
5	**Final Stages**				
5.1	Hardware and Software Integration	2/17/25	2/28/25	11	80%
5.2	Debugging System	3/3/25	3/14/25	11	80%
5.3	Preparing Expo Components	3/3/25	3/21/25	18	0%
5.4	Finalizing System and Presentation	3/21/25	4/4/25	13	0%



Actual Implementation

The figure below summarizes the major components of the entire system, and how they are connected. The major components are the Control Bin, the Backend, and the Reusable Smart Clothing Tag, the Control Bin communicates to the Backend to receive information about clothing products, such as price and product name. The Control Bin also communicates with Smart Clothing Tags to read which products are currently being bought, and to open the tags once a successful purchase has been completed.



Smart Tag

The smart tag is a device that can automatically be detected and auto-release from a clothing item during self-checkout. This reusable clothing tag includes an actuator-driven locking mechanism, an electronic PCB, and an RFID tag; all enclosed within the 3D-printed case. The functions in which the smart tag will operate in are as follows:

- Manual push button to attach clothing item in clothing pin
- RFID tag, detectable during checkout
- Battery-controlled PCB to receive signal and activate release mechanism
- Boost Converter from 3.7V battery to 5V rated loads
- Actuator to pull the rod locking in the spring-loaded block, which will release the clothing pin from the clothing item

Control Bin

The control bin will act as the checkout for customers and a reprogramming station for workers of clothing stores. The control bin must be able to achieve the following functions:

- Display a user interface for customers and workers to use.

- Read the RFID tag within the smart tag.
- Communicate with the backend to make database requests.
- Send signals to the individual smart tags to trigger them to open after a successful purchase.

These functions are achieved using a Raspberry Pi 4, an RFID scanner (M6E nano long distance UHF sensor), a 434.17 MHz transmitter, a monitor, a keyboard, and a mouse. How the above functions are implemented is included in the **Implementation details** section.

Backend

The backend is responsible for maintaining the product database. This involves functions to:

- Add a smart tag (with corresponding product information) to the database.
- Update information relating to a specific smart tag within the database.
- Process incoming requests from the control bin.

Refer to the **Implementation details** for further information.

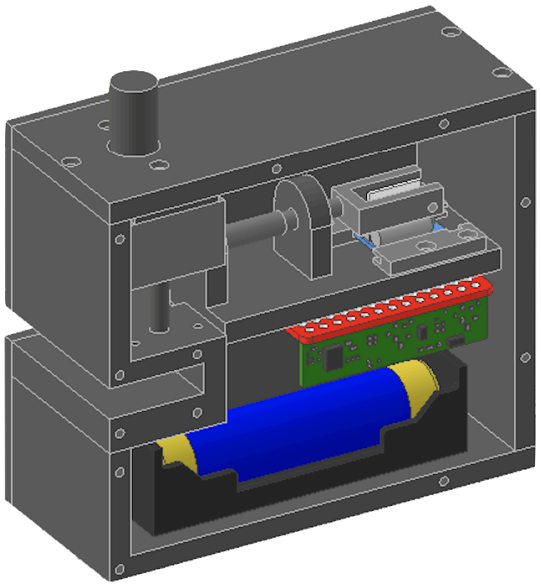
Implementation Details

Hardware

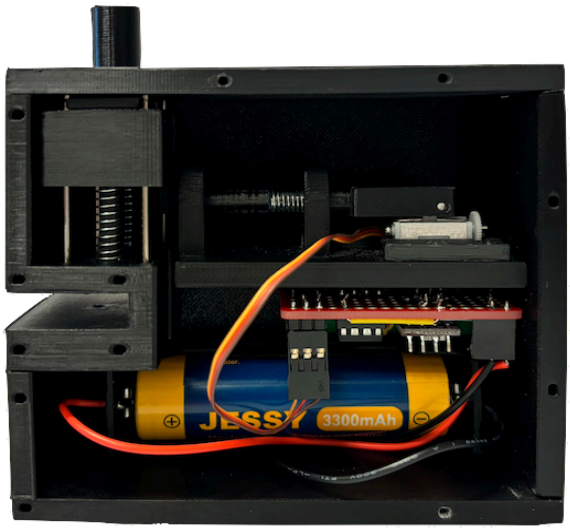
Smart Tag Design

The smart tag consists of a linear actuator controlled by a custom PCB and powered by a battery. The actuator mechanism connects to 3D printed parts and springs to form a mechanical locking/unlocking mechanism to control the state of the pin. An RFID tag is also inside the smart tag to identify the item the tag is attached to. All of these components are enclosed inside a custom 3D printed casing.

CAD Model of Smart Tag



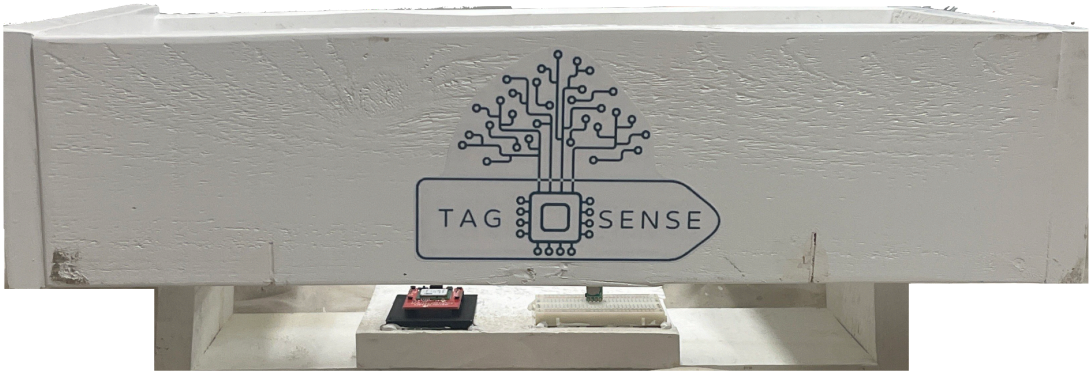
Physical Prototype of Smart Tag



Control Bin Design

The control bin is made of a wooden box for clothing items to be placed inside and identifiable through the smart tag. Below the bin lies the Raspberry Pi, RFID transmitter, and the RFID receiver. Smart tags placed in the control bin can be checked out by the customer after paying and added to the system or read/write to the database through the admin functionalities.

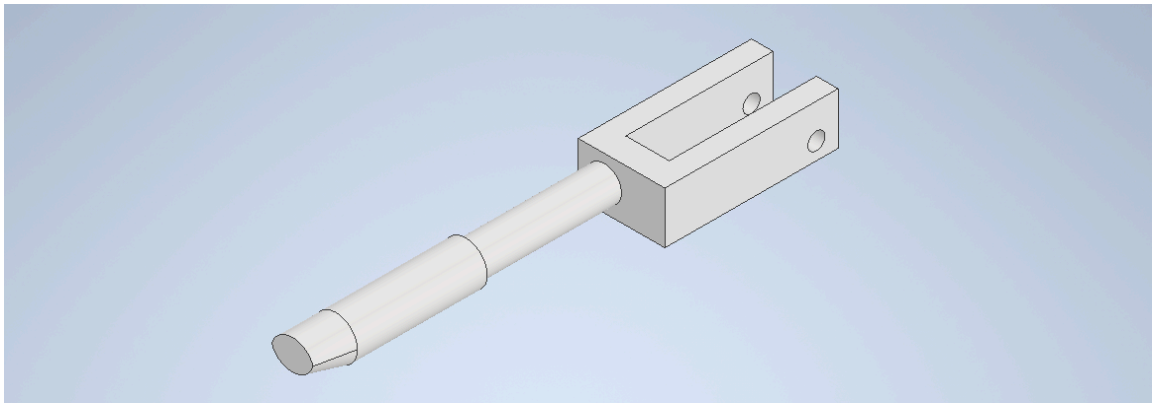
Physical Prototype of Control Bin



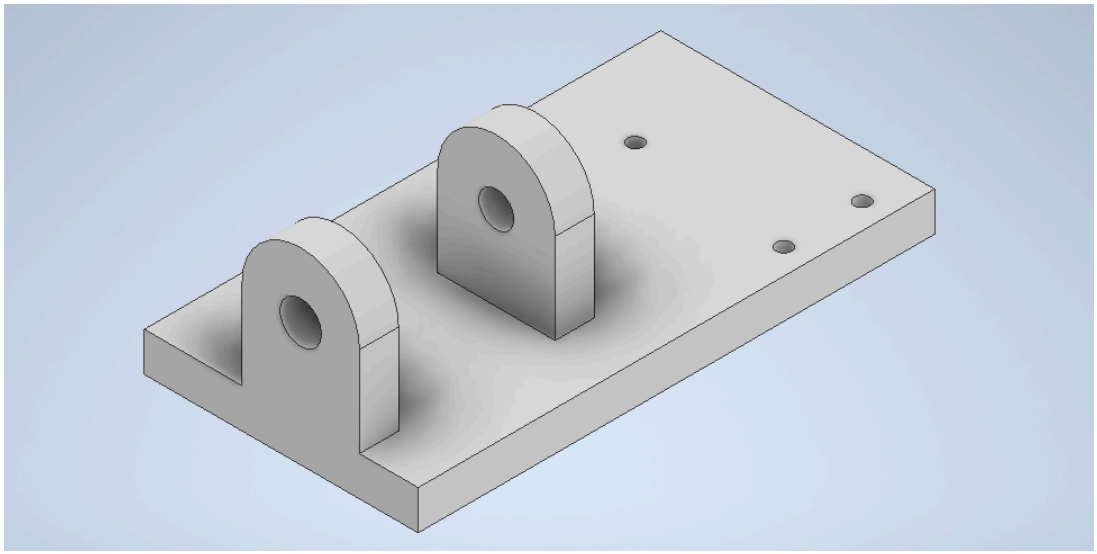
Locking Mechanism

The locking mechanism uses spring-loaded parts and a linear actuator to achieve its functionality. The parts used are, a part with a clothing pin that moves vertically called lockButton, a part that moves horizontally in and out of the lockButton to keep the lockButton in the lowest state called plunger, and two guiding parts one for the lockButton, called caseLock, and one for the plunger, called mountingPlate.

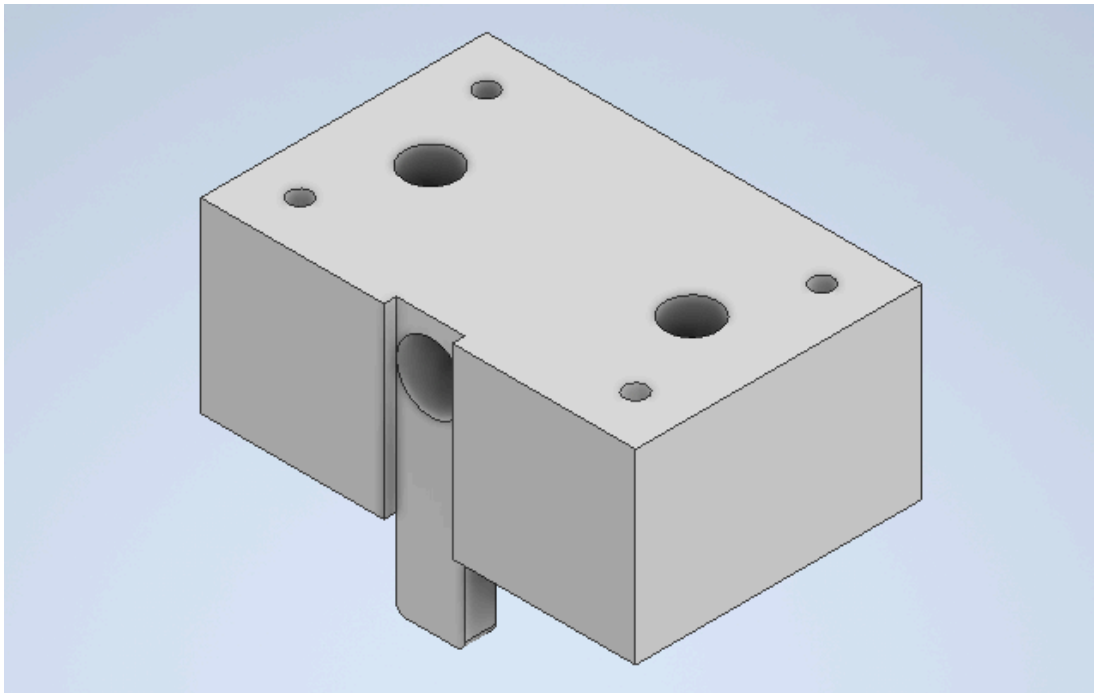
Plunger



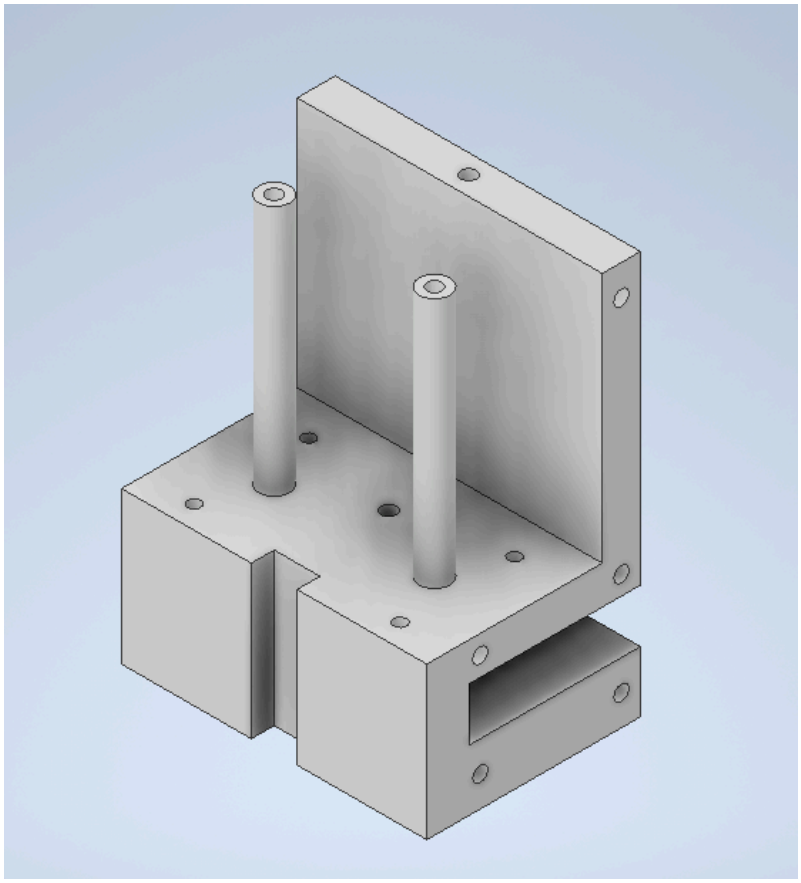
MountingPlate



LockButton

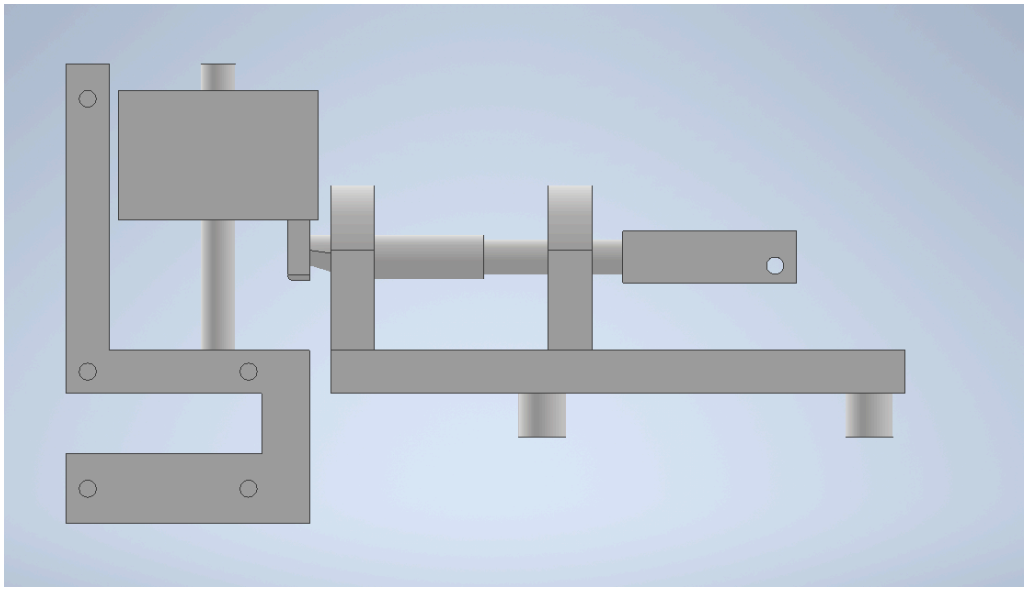


CaseLock

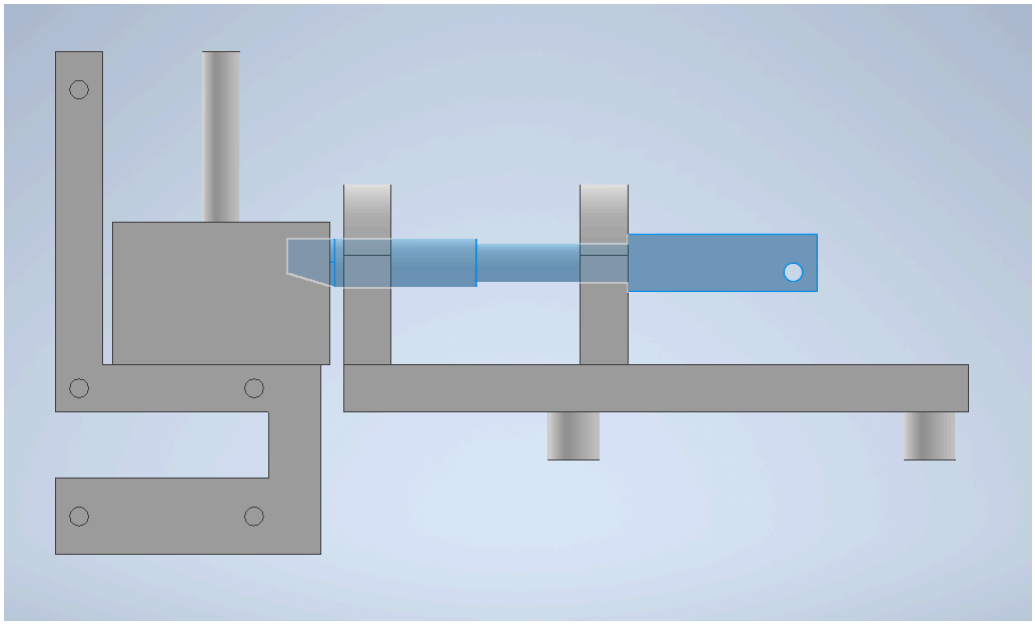


When the lockButton is in the highest position, or the unlocked state, it prevents the plunger from moving to the left to lock the mechanism. When the lockButton is pushed all the way down, there is a hole to allow the plunger to move into the locked position, preventing the lockButton from moving back up and securing the pin that should be at the bottom of the lockButton. A linear actuator installed on the mounting plate and attached to the plunger will be activated to unlock the mechanism by pulling the plunger out of the locked position, allowing the lockButton to move back to the unlocked position.

Unlocked state



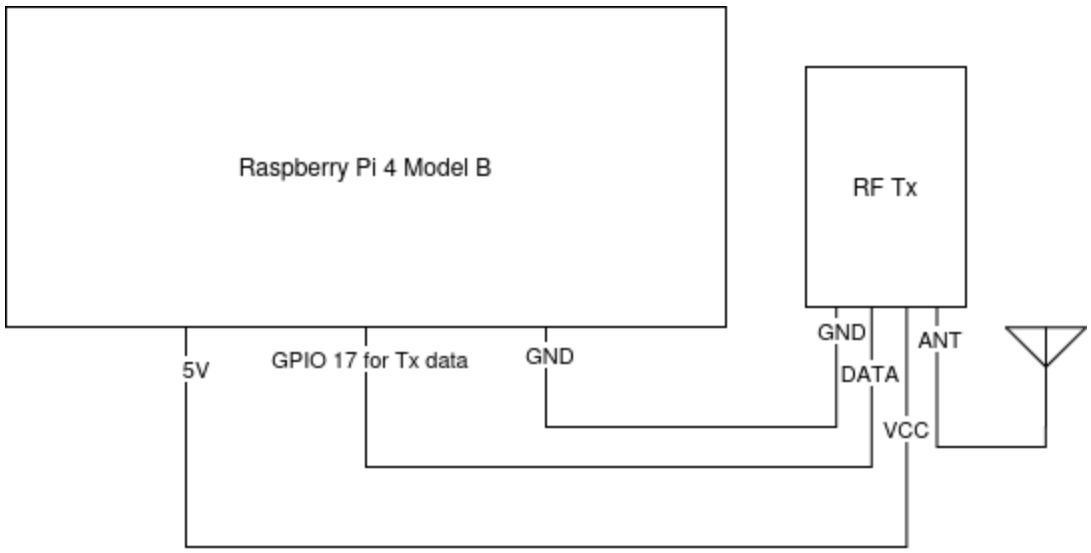
Locked state



Actuator Signaling Circuits

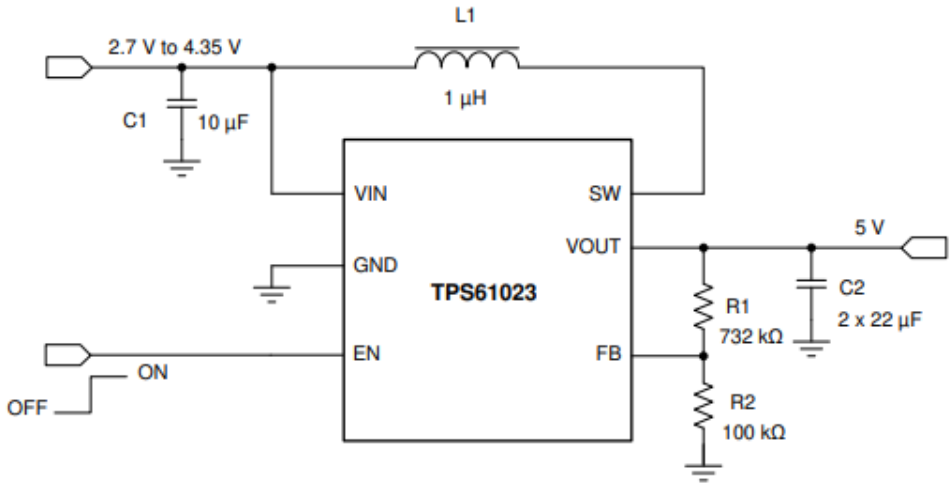
The actuator signaling, for opening the smart tags after a successful purchase, requires three circuits: a transmission circuit in the control bin to send the signal, a receiving and actuating circuit on the smart tag itself, and a flashing circuit to load the ATtiny85 microcontroller with the receiving/actuating program.

Here is a circuit diagram of the transmission circuit:

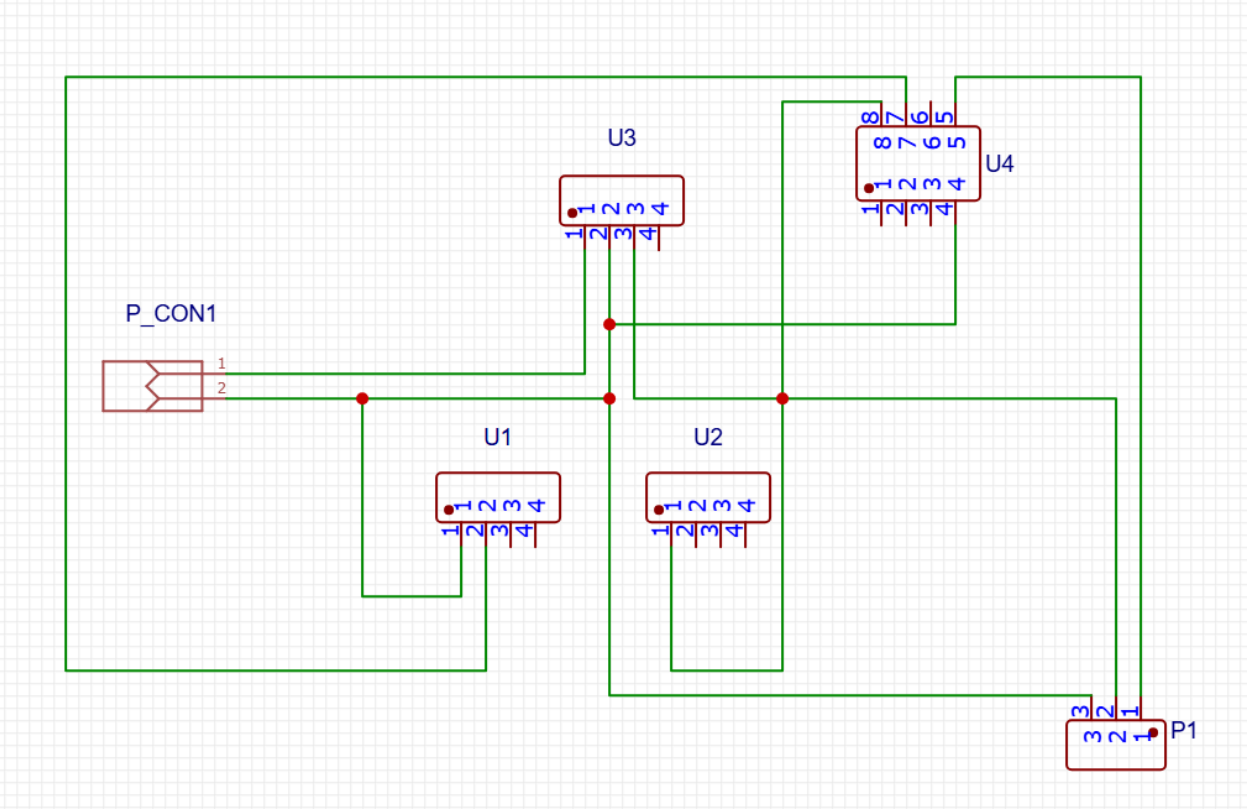


The actuator signal transmission circuit is relatively simple. The Raspberry Pi must provide a 5V power supply and ground to the RF transmitter. It also provides the data to transmit on GPIO 17. Lastly, an antenna is connected to the RF transmitter to increase the signal quality sent out.

Boost Converter utilizing an Adafruit 4654 and TPS61023 with an input voltage of 3.7V:

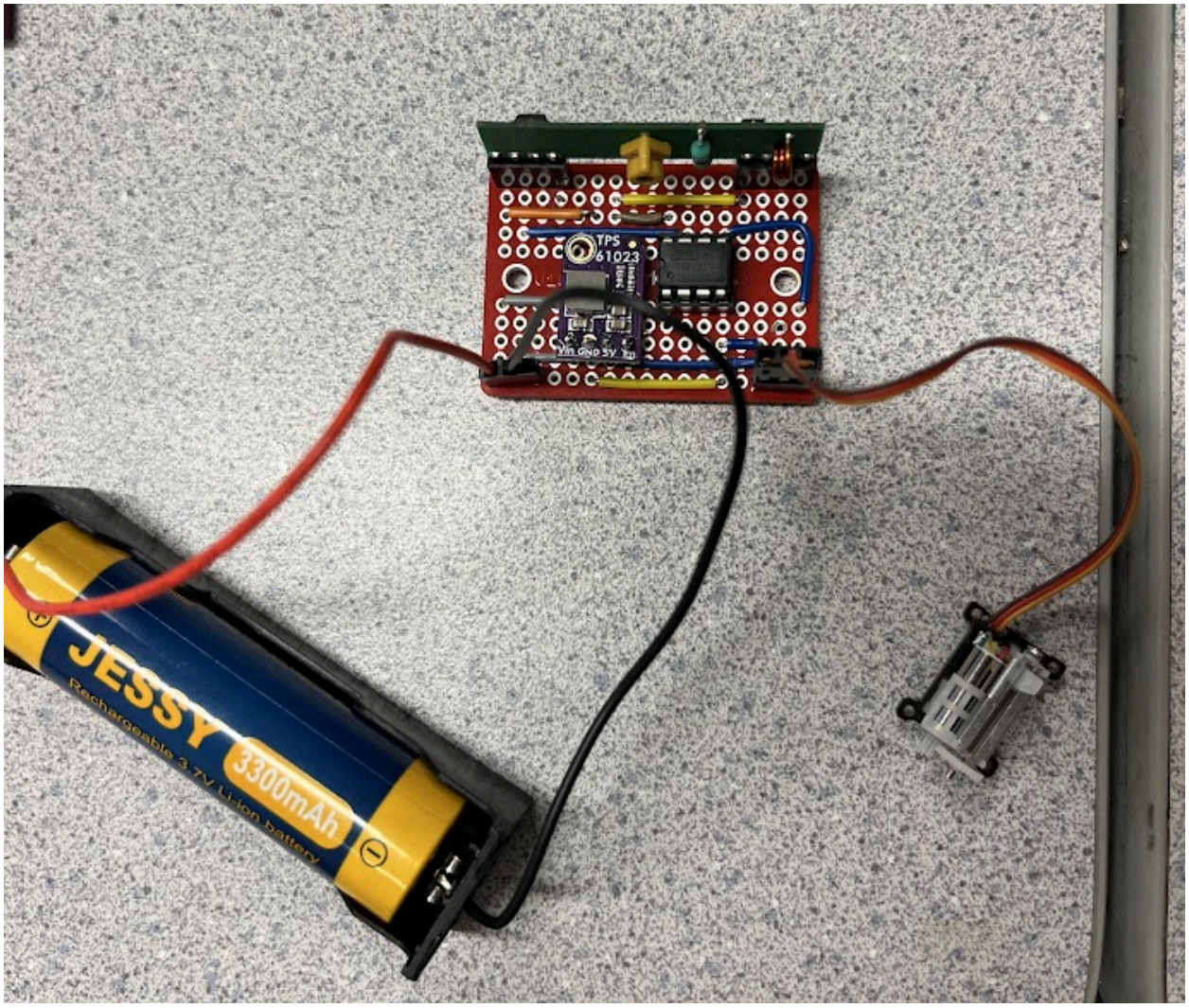


Circuit diagram of the receiving and actuating circuit:



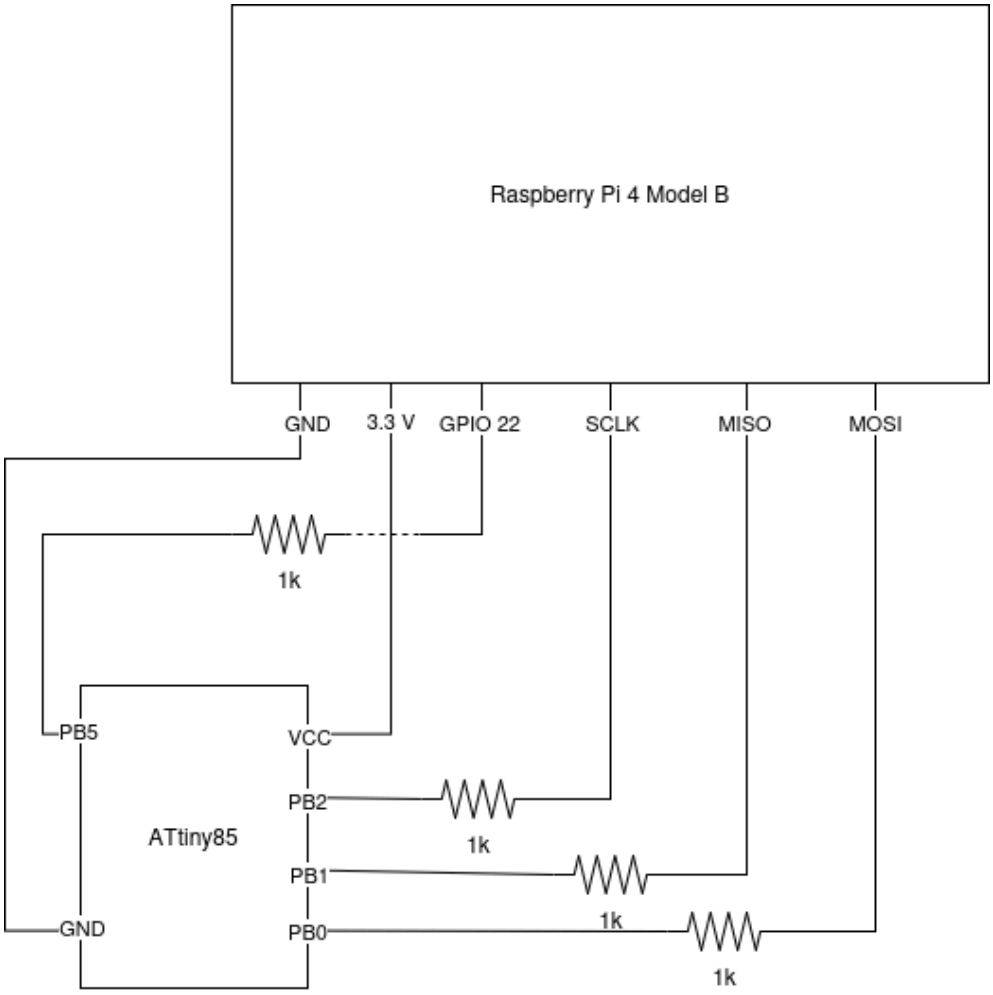
ID	Name	Designator	Manufactuer	Manufacturer PN
1	Receiver	U1& U2	WENSHING	RWS-371-6
2	MiniBoost	U3	Adafruit	4654
3	ATtiny IC	U4	Atmel	ATtiny25
4	Linear Actuator	P1	Vikye	B07VDPFRJ1
5	Battery Holder	P_CON1	MPD	BH-18650-W

The above drawing and BOM highlight the PCB layout of the tag. A battery is connected to P_CON1 at 3.7V, and the system connects at U3, which boosts the power to 5V. Pin 3 of U3 supplies the entire system with the 5V needed, U2 (Pin 1), U4 (Pin 8), and P1 (Pin 2). All loads are also connected to a ground, which ends at P_CON1 (Pin 2), U3 (Pin 2), U4 (Pin 4), U1 (Pin 1), and P1 (Pin 3). The receivers are U1 and U2, which send a signal from pin 2 of the device to pin 7 of U4. U4 is the ATtiny, which essentially works as an IC for the system. If the proper signal is received at pin 7, it will output a signal to pin 5, enabling the linear actuator (P1).



The above image reflects the physical PCB board created to fulfill the design.

Here is a circuit diagram of the ATtiny85 flashing circuit:



The ATtiny85 is flashed over SPI and scheduled reset signals sent from the Raspberry Pi's GPIO 22 pin.

Software

Actuator Signaling Code

This section will show and describe the code written so far to implement the actuator signaling. This includes two functions: one run on the Raspberry Pi to send a signal, and another run on the ATtiny85 microcontroller within the smart tag to receive and process the signal. Here are the contents of the file which sends the open signal from the Raspberry Pi, ‘actuator_signal_send.cpp’:

```
#include "actuator_signal_send.h"

int send_actuator_code_16bit(uint16_t code) {
        //Initialize WiringPi for BCM GPIO numbering scheme
        wiringPiSetupGpio();

        //Initialize RPi as radio controller
        RCSwitch mySwitch = RCSwitch();

        //Connect transmitter
        mySwitch.enableTransmit(GPIO_PIN);

        //Set pulse length.
        mySwitch.setPulseLength(1000);

        //Set number of transmission repetitions.
        mySwitch.setRepeatTransmit(5);

        //Send code
        mySwitch.send(code, SEND_LEN_BITS);

        return 0;
}
```

The function ‘send_actuator_code_16bit’ makes use of the rc-switch and WiringPi libraries. The rc-switch library deals with the communication between the Raspberry Pi and the 434.17 MHz ASK RF transmitter. WiringPi handles the configuration of the GPIO pins. The function first initializes WiringPi for the BCM GPIO numbering scheme. Next, it will initialize an RCSwitch object representing the memory used by the Raspberry Pi to handle the radio controller. Then, the transmitter is enabled on a predefined GPIO_PIN (from ‘actuator_signal_send.h’). We have chosen to use the GPIO pin 17, or the physical pin 11. Further transmit configurations include setting the pulse length to 1 ms, and setting the number of transmission repetitions to 5. Due to the timing constraints brought about by the resource restrictive ATtiny85, the pulse length was increased as well as the number of repeat transmissions to ensure that the ATtiny85 will always be able to decipher a received code. Lastly, ‘actuator_signal_send.cpp’ will send the input code over a predefined 16 bits.

Here is the file containing the program flashed on the ATtiny85 for testing with the 434.17 MHz ASK RF receiver and the linear servo motor actuator, ‘rx_act_testing.ino’. This program will be very similar to the final program flashed, with some extraneous functionalities for testing removed:

```
#include <RCSwitch.h>
#include <SoftwareServo.h>

#define RX_PIN 0 //Use PB2 (INT0) for receiving
#define SERVO_PIN 0 //PB0 (physical pin 5)
#define SERVO_UPDATE_DELAY 10
#define EXPECTED_CODE 11981

#define LED_PIN 1 //Use PB1 to flash LED

RCSwitch tiny_switch = RCSwitch();
SoftwareServo lin_act;

void setup() {
 //Run clock at the full 8MHz
 CLKPR = (1 << CLKPCE); //Enable changes to the clock prescaler
 CLKPR = 0; //Set the prescaler to 0 (no clk division)

 //Configure RF receiver
 tiny_switch.enableReceive(RX_PIN);   // Use pin 2 for receiving

 //Initialize linear actuator servo motor
 lin_act.attach(SERVO_PIN);
 lin_act.setMinimumPulse(832);

 //Configure LED_PIN as output
 pinMode(LED_PIN, OUTPUT);

 //Flash LED to confirm setup complete
 for (int i=0; i<3; i++) {
         blink_led();
 }
}

void blink_led() {
 digitalWrite(LED_PIN, HIGH);
 delay(500);
 digitalWrite(LED_PIN, LOW);
 delay(500);
}
```

```
void actStepTo(int pos) {
  //Function for changing PWM of servo motor to reach desired position (pos).
  //pos is a value between 0 and 180 degrees.
  lin_act.write(pos);
  delay(SERVO_UPDATE_DELAY);
  SoftwareServo::refresh();
}

void openTag() {
  //Function to open tag using servo motor.
  int start_angle = 1;
  int end_angle = 180;

  for(int pos = start_angle; pos <= end_angle; pos += 1) {
          actStepTo(pos);
  }
  delay(1000);
  for(int pos = end_angle; pos >= start_angle; pos -= 1) {
          actStepTo(pos);
  }
}

void loop() {
  if (tiny_switch.available()) {
          //Non-zero code received over RF. Fetch it here
          uint16_t code = tiny_switch.getReceivedValue();

          for (int i=0; i<2; i++) {
          blink_led();
          }

          //Move linear actuator (servo motor) if expected code is received
          if (code == EXPECTED_CODE) {
          openTag();
          }
          else {
          actStepTo(code);
          }

          tiny_switch.resetAvailable();
  }
}
```

‘rx_act_testing.ino’ makes use of the rc-switch and SoftServo libraries. As with the Raspberry Pi, rc-switch handles the RF communication. SoftServo provides functions to control the linear servo motor actuator. First, we define the pin to use for receiving signals for the RF receiver (RX_PIN), the pin for outputting the servo motor’s PWM signal (SERVO_PIN), the pin for controlling an LED (LED_PIN), and the expected received code that will trigger the actuator to open the tag (EXPECTED_CODE). After these are defined, under the setup function, the ATtiny85’s internal clock is set to run at 8 MHz, which is essential for the RF communication timing to work. Also, the RF receiver, servo motor, and LED are configured. The LED flashes three times to show that the setup is complete. The function ‘blink_led’ defines how the ATtiny85 will blink the LED with the LED pin. The functions ‘actStepTo’ and ‘openTag’ work together to define how the ATtiny85 will move the actuator when opening the tag. Finally, the loop function will run the main program. In this loop, the ATtiny85 waits for a code to be received over RF. Once a code is received, it blinks the LED to show that it received a code. If the code matches the expected code, it will open the tag, otherwise it will step to the position given by the code (this function is for testing purposes only and will simply be deleted for the final program flash; meaning the ATtiny85 does nothing if an incorrect code is received). In the final system, the LED code will be removed as it is only added to provide visual feedback while testing.

The following Makefile alongside the avrdude library is used to flash the ATtiny85 with ‘rx_act_testing.ino’ from the Raspberry Pi after the compiled binary (.hex file) is exported from the Arduino IDE:

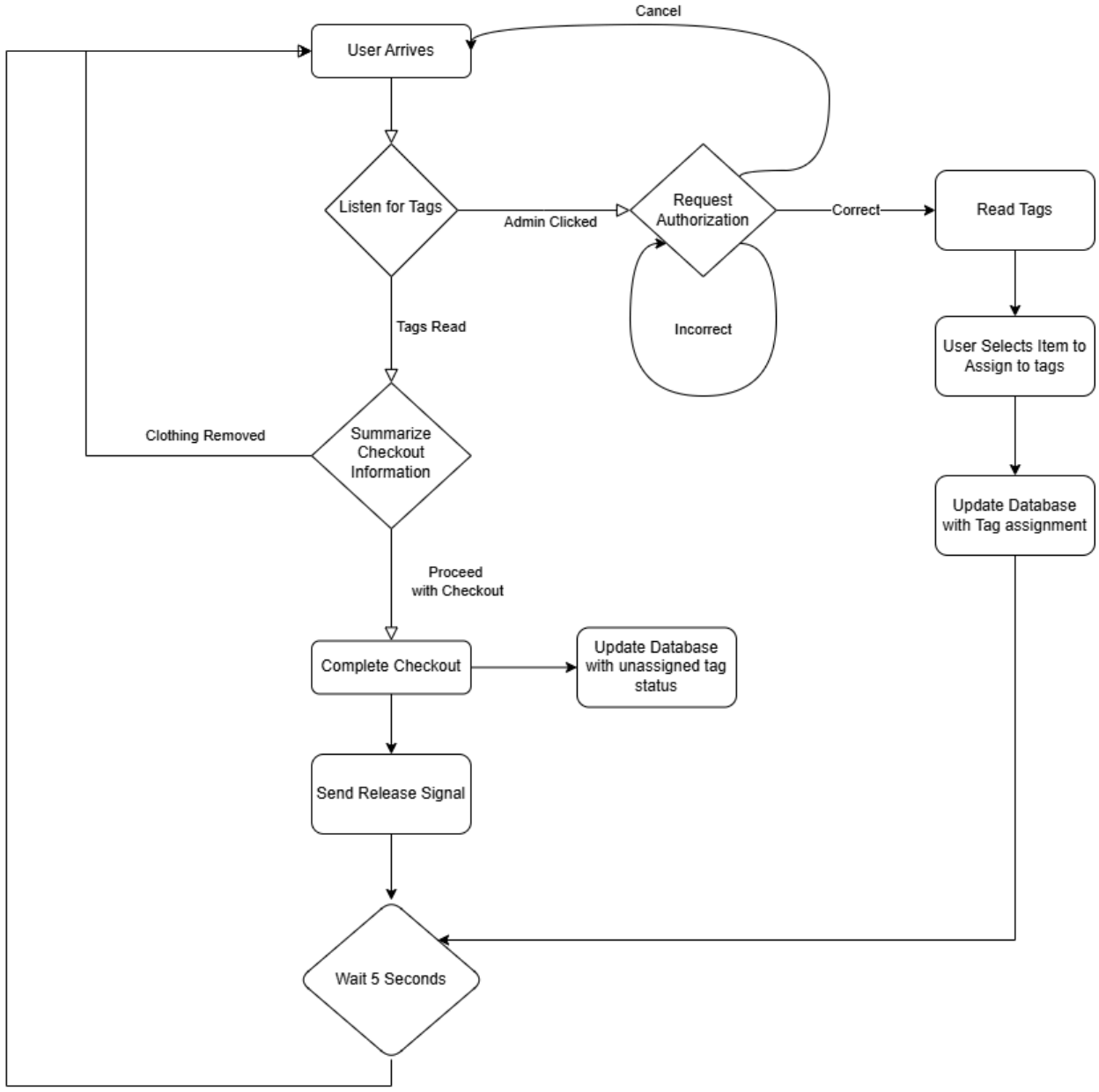
```
install :
	sudo gpio -g mode 22 out
	sudo gpio -g write 22 0
	sudo $(AVRDUDE) -p $(AVRDUDEMCU) -P /dev/spidev0.0 -c linuxspi -b 10000 -U
flash:w:$(HEXFILE)
	sudo gpio -g write 22 1

noreset :
	sudo $(AVRDUDE) -p $(AVRDUDEMCU) -P /dev/spidev0.0 -c linuxspi -b 10000 -U
flash:w:$(HEXFILE)

fuse :
	sudo gpio -g mode 22 out
	sudo gpio -g write 22 0
	sudo $(AVRDUDE) -p $(AVRDUDEMCU) -P /dev/spidev0.0 -c linuxspi -b 10000 -U
lfuse:w:0x62:m -U hfuse:w:0xdf:m -U efuse:w:0xff:m
	sudo gpio -g write 22 1
```

Command Line Interface

The command line interface is a preliminary implementation to demonstrate the capabilities of the full checkout system. It follows the following system flow chart:



Before entering into this loop, the program must first initialize the various tools that will be used within it. These tools include the ThingMagic RFID reader and the database. We also initialize various global variables that will be used throughout the software system.

```
extern TMR_Reader r, *rp;
extern TMR_Status ret;
extern TMR_ReadPlan plan;
extern TMR_Region region;
extern int readpower;
extern uint8_t i;
extern uint8_t buffer[20];
extern uint8_t *antennaList;
extern uint8_t antennaCount;
extern TMR_TRD_MetadataFlag metadata;
extern char string[100];
extern TMR_String model;
char input;
int admin_request;
int close_request = 0;
sqlite3 *db;
```

In our main system we begin to initialize our system variables. We begin by initializing our database and also our RFID reader.

```
int main(int argc, char *argv[]) {

    // Initialize system variables
    admin_request = 0;
    rp = &r;
    int system_status;
    // Initialize Database variables
    int sql_status;
    sql_status = sqlite3_open("RFID_SYSTEM_DB.db", &db);

    // Initialize the reader
    reader_init();

    if(sql_status != SQLITE_OK) {

        //Get error
        const char * errmsg = sqlite3_errmsg(db);
        printf("ERROR OPENING DATABASE, ERROR:\n\n %s\n", errmsg);
        printf("SHUTTING DOWN\n");

        //Close system
        sqlite3_close(db);
        return 1;
```

```
}

// Create the table if it is not created yet
sql_status = create_table(db);
if (sql_status != SQLITE_OK){

    //Get error
    const char * errmsg = sqlite3_errmsg(db);
    printf("ERROR CREATING TABLE, ERROR:\n\n %s\n", errmsg);
    printf("SHUTTING DOWN\n");

    //Close system
    sqlite3_close(db);
    return 1;
}
```

The required connections are complete and the system can now begin the loop. To listen to both the reader and the admin interaction we will create two threads. When these threads complete, we will return to the main loop and perform the required tasks.

```
while(1) {

    //Listen for User and Listen for admin requests
    pthread_create(&RFID_scan, NULL, RFID_thread, NULL);
    pthread_create(&input_scan, NULL, input_thread, NULL);



    pthread_join(RFID_scan, NULL);
    pthread_join(input_scan, NULL);
```

The Definitions for each of the threads are shown below. The RFID thread continuously scans until it hears or sees a successful read. It also listens to the potential exit or admin flags to terminate the thread from proceeding. The input thread, listens for specific characters to initiate either an admin request or an exit program request. After this, the thread terminates.

```
void* RFID_thread(void* vargp)
{
 do {

          printf("LISTENING FOR TAGS...\n");
          fflush(stdout);

        //Read tags
        ret = TMR_read(rp, 500, NULL);

        if (TMR_ERROR_TAG_ID_BUFFER_FULL == ret)
        {
          /* In case of TAG ID Buffer Full, extract the tags present
          * in buffer.
          */
          #ifndef BARE_METAL
             fprintf(stdout, "reading tags:%s\n", TMR_strerr(rp, ret));
          #endif /* BARE_METAL */
        }
        else
        {
          checkerr(rp, ret, 1, "reading tags");
        }

        //Do a 0.5 second delay in between reads
        delay(2200);
    }while(TMR_SUCCESS != TMR_hasMoreTags(rp) && admin_request != 1 && close_request ==
0);

   printf("Tags or Admin Request!\n");
   fflush(stdout);
}
```

```
void* input_thread(void* vargp)
{
   delay(1000);
   printf("LISTENING FOR ADMIN REQUEST\n");
   fflush(stdout);
   scanf(" %c", &input);
   if(input == 'r'){
      admin_request = 1;
   }
   else if(input == 'x'){
      close_request = 1;
   }
}
```

When these threads complete the system will enter into the corresponding path depending on which flags have been set. The two flags that can be set are the admin request and the close request flags. When an admin request flag is seen, we enter into the admin authentication, followed by the admin sequence. the other flag is a close request which simply breaks the system loop and exits the application. When neither of the flags are set, we have tags sensed and proceed to the checkout sequence.

```
      if(admin_request == 1) {

         //Go down admin path
         admin_request = 0;
         system_status = authenticate();

         //After proper authentication, request input and perform task
         if(system_status == 1) {
            handle_admin();
         }

      }
      else if(close_request ==1) {
         break;
      }
      else {
         //Go down scanned path
         admin_request = 0;
         handle_checkout();
      }
   }
```

These are all within the loop, that will run indefinitely. The authentication function asks for a user input, and verifies if the input matches the password. This is a constant currently set in our code.

```
int authenticate() {

   int input;
   delay(1000);
   printf("Please input the admin password:\n");
   fflush(stdout);
   delay(1000);
   scanf(" %d", &input);
   if(input == 1234){
      printf("Correct Password!\n");
      fflush(stdout);
      return 1;
   }else {
      printf("Incorrect Password!\n");
      fflush(stdout);
      return 0;
   }
}
```

If the authentication is correct, we then proceed to admin selection screen which allows the user to select which process they would like to complete.

```
void handle_admin() {

   char input;

   //Handle user input and send to correct path
   while(1){
      printf("Select the process you would like to complete:\n");
      printf("\tAdd new tags (a)\n\tAssign existing tags (u)\n\tExit Admin (x)\n");
      fflush(stdout);
      delay(1000);
      scanf(" %c", &input);

      if(input == 'a'){
         handle_add_tags();
         break;
      }
      else if(input == 'u'){
         handle_update_tags();
         break;
      }
```

```
        else if(input == 'x'){
            printf("Exiting Admin\n");
            fflush(stdout);
            break;
        }
        else{
            printf("INVALID INPUT\n");
            fflush(stdout);
        }
    }

}
```

Within this loop we have two functions that handle the two processes the admin can complete. They can either add tags into the system or they can update the item the tags correspond to in the system. The add tags function prepares the user to place the tags in the correct place, reads the tags, and pushes a database update for each tag. The update sequence follows the same pattern but also asks for the item ID and does a different request. The functions are shown on the following pages.

```
void handle_add_tags(){

   char input;

   //Prepare the tags
   printf("Please place the tags in the bin that you would like to add...\n");
   printf("When ready, initiate the request py pressing a key and enter\n");
   fflush(stdout);
   delay(1000);
   scanf(" %c", &input);

   //Read the tags
   TMR_read(rp, 500, NULL);

   //Add the tags to the database as null values
   while (TMR_SUCCESS == TMR_hasMoreTags(rp))
   {
      TMR_TagReadData trd;
      char idStr[128];

      ret = TMR_getNextTag(rp, &trd);
      checkerr(rp, ret, 1, "fetching tag");

      TMR_bytesToHex(trd.tag.epc, trd.tag.epcByteCount, idStr);
      add_tag(db, idStr);
      printf("Tag added: %s", idStr);
   }

   printf("Tags added to database!\n");
   fflush(stdout);

}
```

```
void handle_update_tags(){

   char input;
   char itemID[10];

   //Prepare the tags
   printf("Please place the tags in the bin that you would like to update...\n");
   delay(1000);
   printf("When ready, initiate the request py pressing a key and enter\n");
   fflush(stdout);
   delay(1000);
   scanf(" %c", &input);

   // Determine the Item to change them to
   printf("Input the ItemID that you would like to assign them to");
   fflush(stdout);
   delay(1000);
   scanf(" %s", itemID);

   //Read the tags
   TMR_read(rp, 500, NULL);

   //Add the tags to the database as null values
   while (TMR_SUCCESS == TMR_hasMoreTags(rp))
   {
      TMR_TagReadData trd;
      char idStr[128];

      ret = TMR_getNextTag(rp, &trd);
      checkerr(rp, ret, 1, "fetching tag");

      TMR_bytesToHex(trd.tag.epc, trd.tag.epcByteCount, idStr);
      update_tag_item(db, itemID, idStr);
   }

   printf("Tags updated to itemID = %s\n", itemID);
   fflush(stdout);
}
```

That is the extent of the current admin functionality. The checkout sequence uses the previously scanned tags to make a database request to determine the assigned item. The item price and descriptions are printed out and displayed on teh screen. The user is then prompted to confirm or cancel the checkout. If the checkout is confirmed. the system sends out the release signal using the compiled “tx” program releasing the mechanism on the tags.

```
void handle_checkout(){

   char input;
   int price = 0;
   //Collect Tag Data
   while (TMR_SUCCESS == TMR_hasMoreTags(rp))
   {
      TMR_TagReadData trd;
      char idStr[128];
      char itemID[20];

      ret = TMR_getNextTag(rp, &trd);
      checkerr(rp, ret, 1, "fetching tag");

      TMR_bytesToHex(trd.tag.epc, trd.tag.epcByteCount, idStr);
      get_tag(db, idStr, itemID, sizeof(itemID));
      price += get_item(db, itemID);
   }

   printf("Total: $%d\n", price);
   printf("Proceed with checkout?(y/n)\n");
   scanf(" %c", &input);

   if(input == 'y'){
      printf("Completed Purchase\n");
                  system("./tx");
                  delay(2000);
   }
   else{
      printf("Checkout Cancelled\n");
   }

}
```

This is the complete CLI loop implementation, some details on the utilities used within the program are described below. These details describe some of the database requests, and reader initialization functions that are used within the main loop.

Database

The database is implemented using an sqlite3 database. This database contains two tables to manage the tags and the clothing items. The names of these tables are “Tags” and “Items” respectively. A breakdown of the tables with example values is shown below.

“Tags”

TagID	**ItemID**	**status**
RFID Tag Code	Clothing item ID	Bit corresponding to tag status
“EF200123DA1234”	“1”	0

“Items”

ItemID	**ItemDescription**	**price**
Clothing item ID	Short clothing item description	Item price
“1”	“A Black shirt”	30

There are various functions implemented to interact with the database. These are add tag, update tag, get tag, and get item. They are all implemented using the same format, they only use different SQL commands. We will summarize the difference and show an example using the add_tag function.

```
int add_tag(sqlite3 * db, char tagID[]) {

   int status;
   sqlite3_stmt * statement_ptr;
   char command[200];

   strcpy(command,"INSERT INTO Tags (tagID, itemID, status) VALUES('");
   strcat(command, tagID);
   strcat(command, "', '0', 0)");

   //NEED to work out logic to add tag ID into this string
   status = sqlite3_prepare_v2(db, command, -1, &statement_ptr, 0);

   status = sqlite3_step(statement_ptr);
   status = sqlite3_finalize(statement_ptr);

   return status;
}
```

The command that is used is the INSERT INTO function and we pass in the tag ID and set it into the value string using the strcat() method. These are then executed using the sqlite3 functions. The update tags uses the same format but instead uses:

```
UPDATE Tags SET itemID = {itemID} WHERE tagID = {tagID};
```

This updates the itemID value for the tags. The get tag function uses the following command:

```
SELECT tagID,itemID,status FROM Tags WHERE tagID = {tagID};
```

Finally, the get item tag uses the following command:

```
SELECT itemID,itemDescription,price FROM Items WHERE itemID = {itemID};
```

These functions describe the various interactions we have with our sqlilte3 database.

RFID Reader

The reader is connected using the mercuryAPI built in functions. The process used is documented in their documentation and is followed for the C language. We also use the TMR_Read() function which commits a read plan for the given amount of time for the specific reader. We commonly found that the best timing and power used for the RFID reader is a 0.5s read sequence with a power of 2100cdbm, further testing with the full system implemented will help identify the best values for performance.

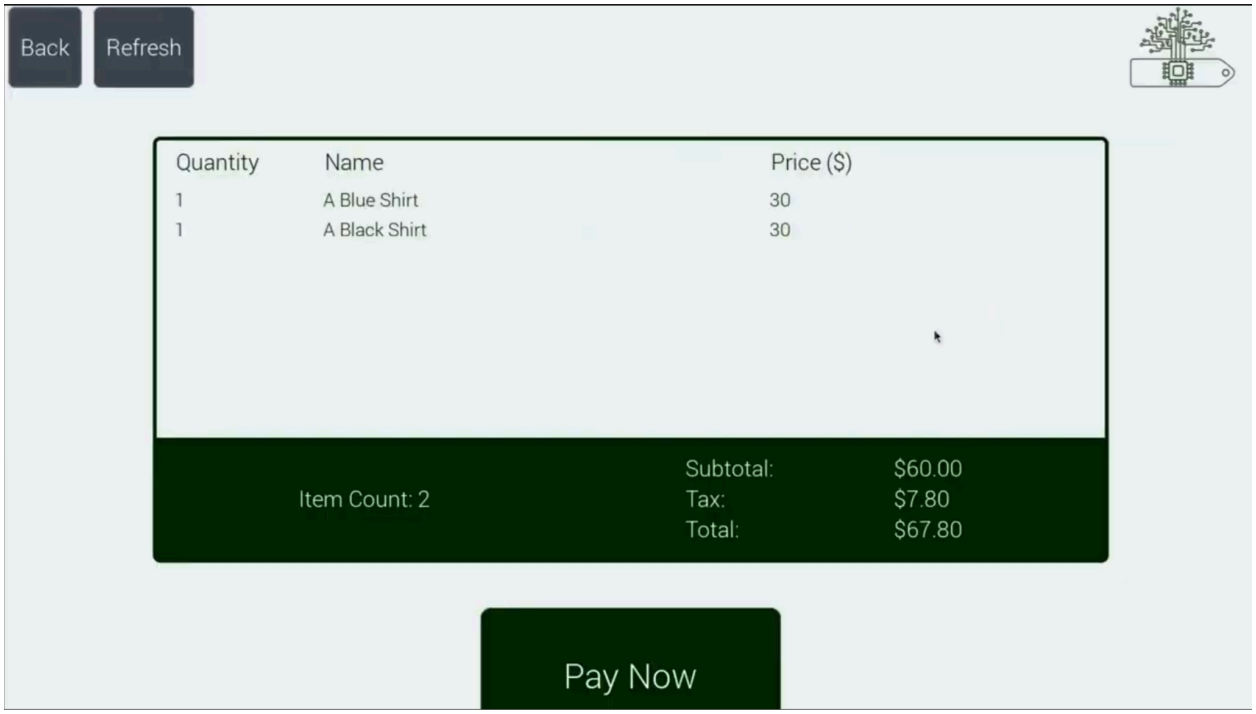
Graphical User Interface

The control bin's Graphical User Interface (GUI) allows customers to purchase clothing and administrators of a retail clothing store to reprogram tags without needing to use a command line interface (CLI). This makes our system much easier to learn how to use.

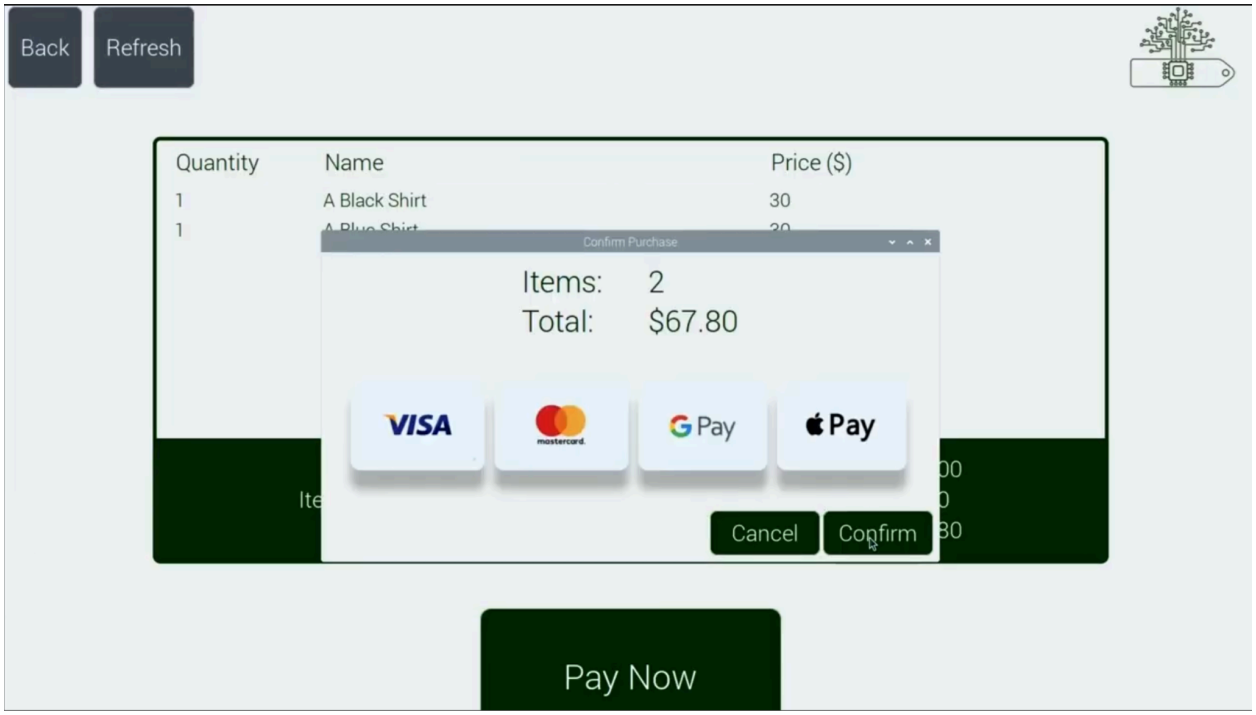
The GUI was created using GTK3, a widget toolkit for creating graphical user interfaces which works very well with our Raspberry Pi 4, the CPU of our control bin. The GUI is composed of multiple pages: a startup page, a checkout page, an admin login page, and an admin page.

The startup page is the page users will be greeted with when starting to use the system. It contains our logo in the top right corner, the TagSense name prominently in the middle, and a large 'Start' button. Here is a screenshot of the startup page:

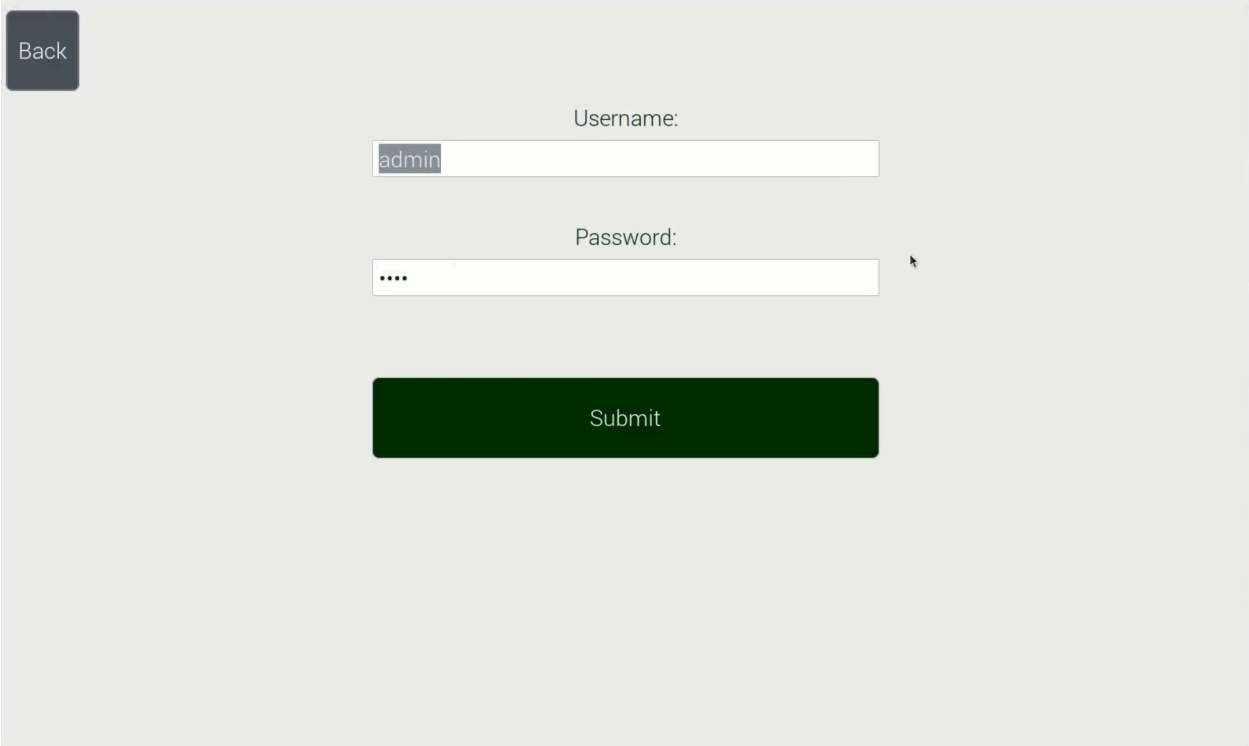
After clicking or tapping the start button, the system moves to the checkout mode. This is where customers can purchase clothing items. As soon as the start button is clicked, the control bin automatically makes a scan and displays any items in the bin. In the example screenshot below, two tags representing a black and a blue shirt were present. The GUI automatically makes database requests to figure out which items are present in the control bin, and displays and calculates the price. In the top left corner. 'Back' and 'Refresh' buttons are present. The 'Back' button can be selected to return back to the startup page. The 'Refresh' button initialises a new RFID tag scan, which will allow the customer to remove and add new items to their total as they see fit. The 'Pay Now' button below the checkout item list can be clicked once the customer is happy with their purchase and they are ready to buy their clothing items.



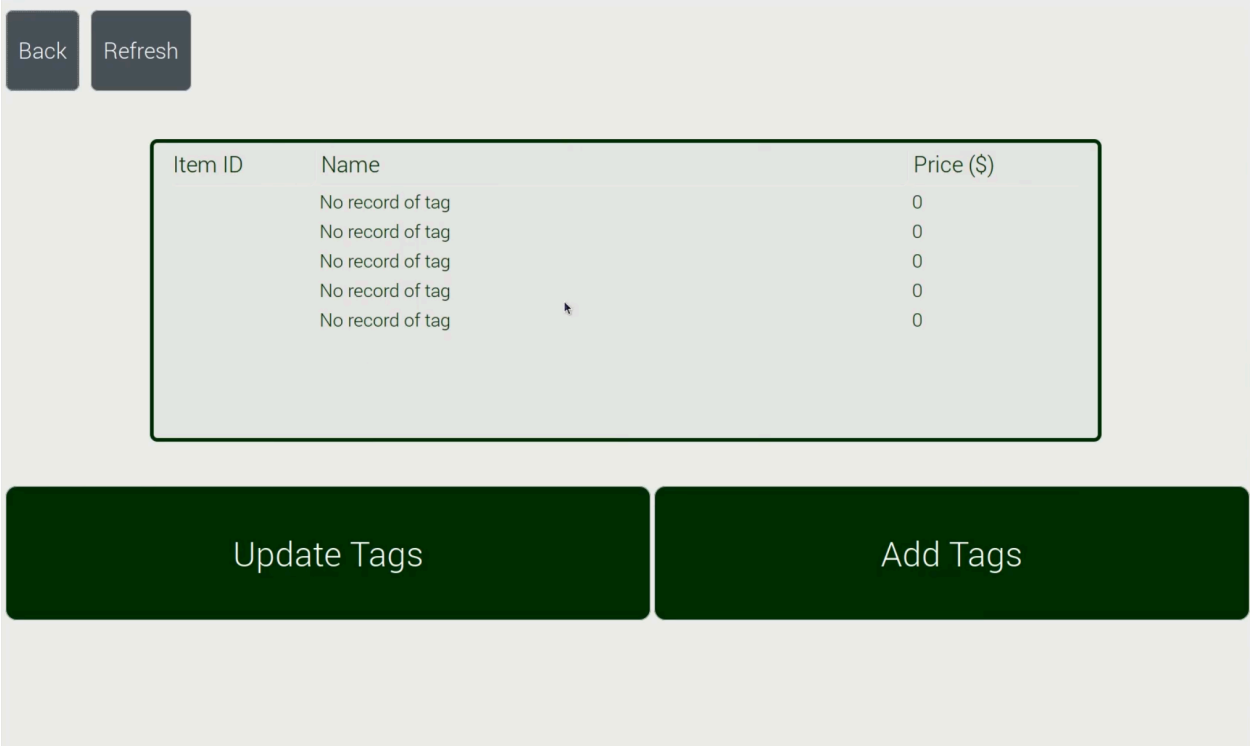
After clicking the ‘Pay Now’, a dialog box appears which allows the customer to complete their purchase. Selecting ‘Cancel’ returns the customer to the checkout page. Clicking ‘Confirm’ completes the purchase, which causes the control bin to send a release signal to the clothing items’ tags and returns the GUI to the startup page. From a customer’s perspective, they complete their purchase, then the tags on their new clothes automatically release themselves and they may take them home. Here is a screenshot of the checkout page with the ‘Confirm Purchase’ dialog box:



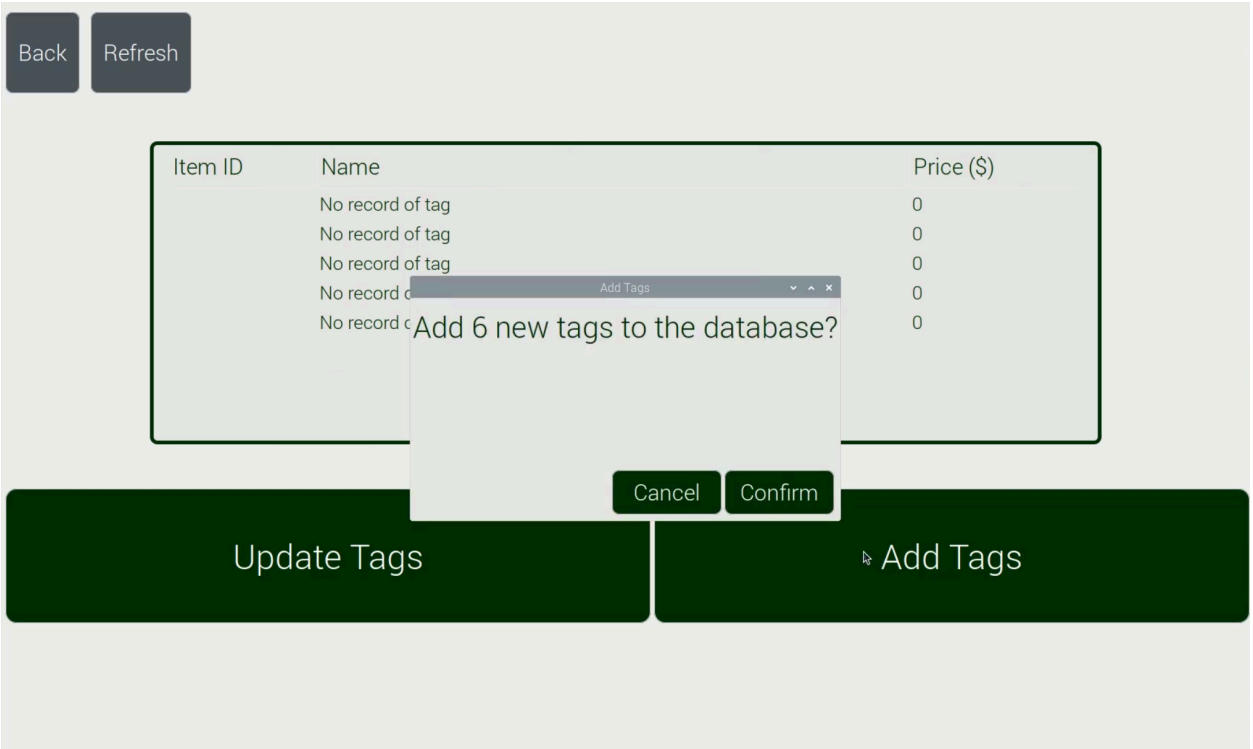
An administrator of the system (a retail store owner or worker) may select the TagSense logo in the upper right of the startup or checkout page to open the admin login page. The admin login page asks for the admin username and password. There is a ‘Submit’ button to attempt an admin login with the input username and password, and a ‘Back’ button to return to the startup page. Here is a screenshot of the admin login page:



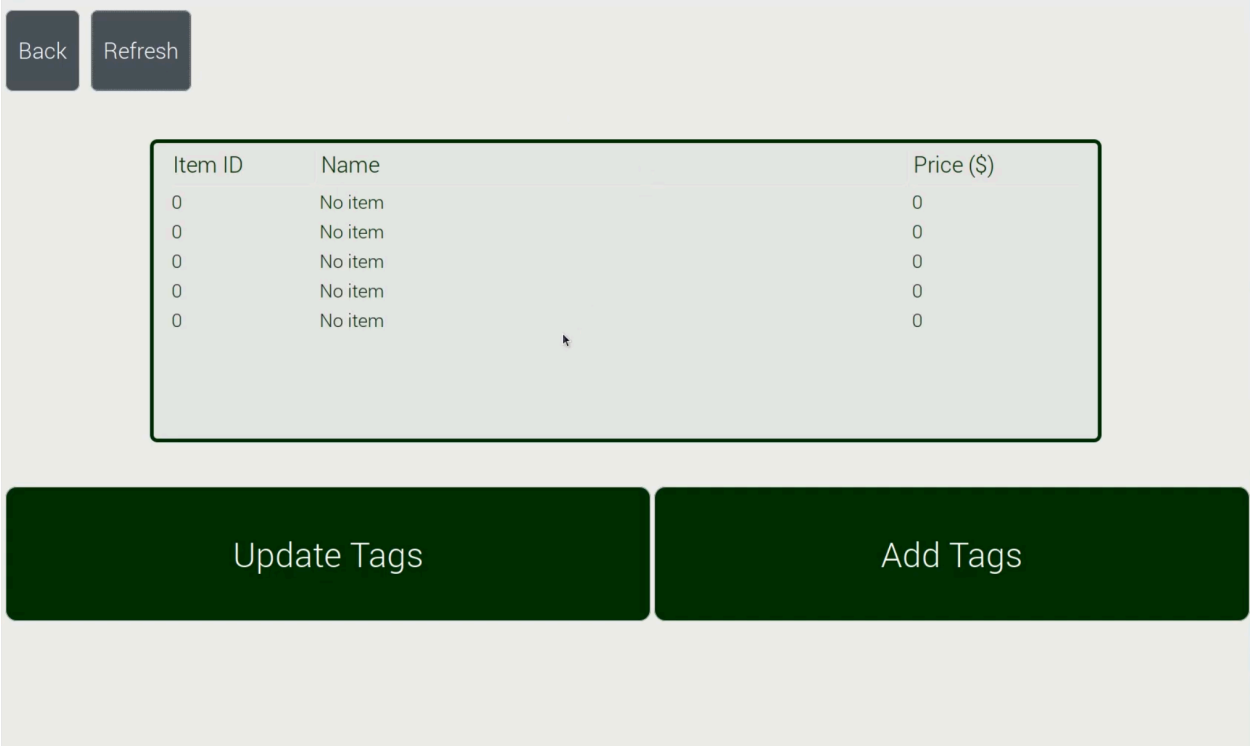
After a successful login, the user will be taken to the admin page. The admin page is similar to the checkout page, except for the ‘Update Tags’ and ‘Add Tags’ buttons. Also, scanned tags not registered within the database will appear here with Name ‘No record of tag’. Here is the admin page with 6 new tags (not in the database) in the control bin:



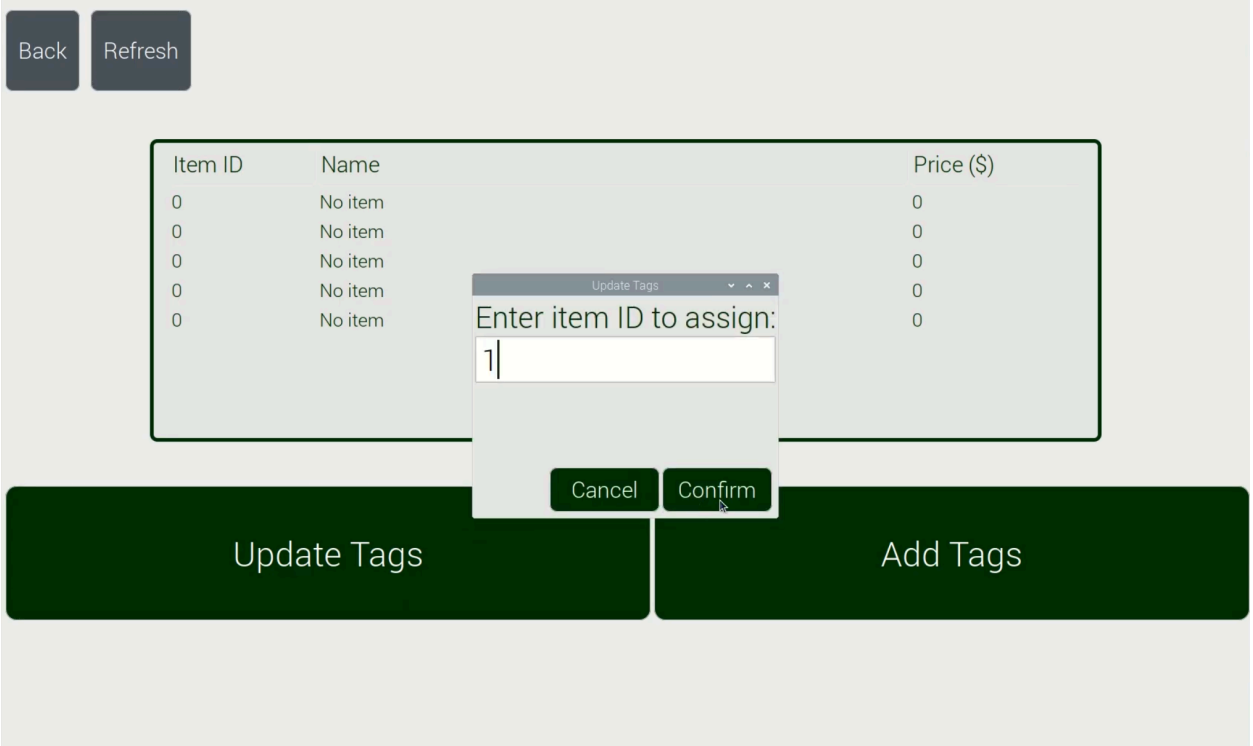
Once ‘Add Tags’ is clicked, the control bin counts the amount of new tags in the checkout, then prompts the user to add the new tags to the database. Selecting ‘Cancel’ does not add the new tags to the database, while selecting ‘Confirm’ will add the new tags:



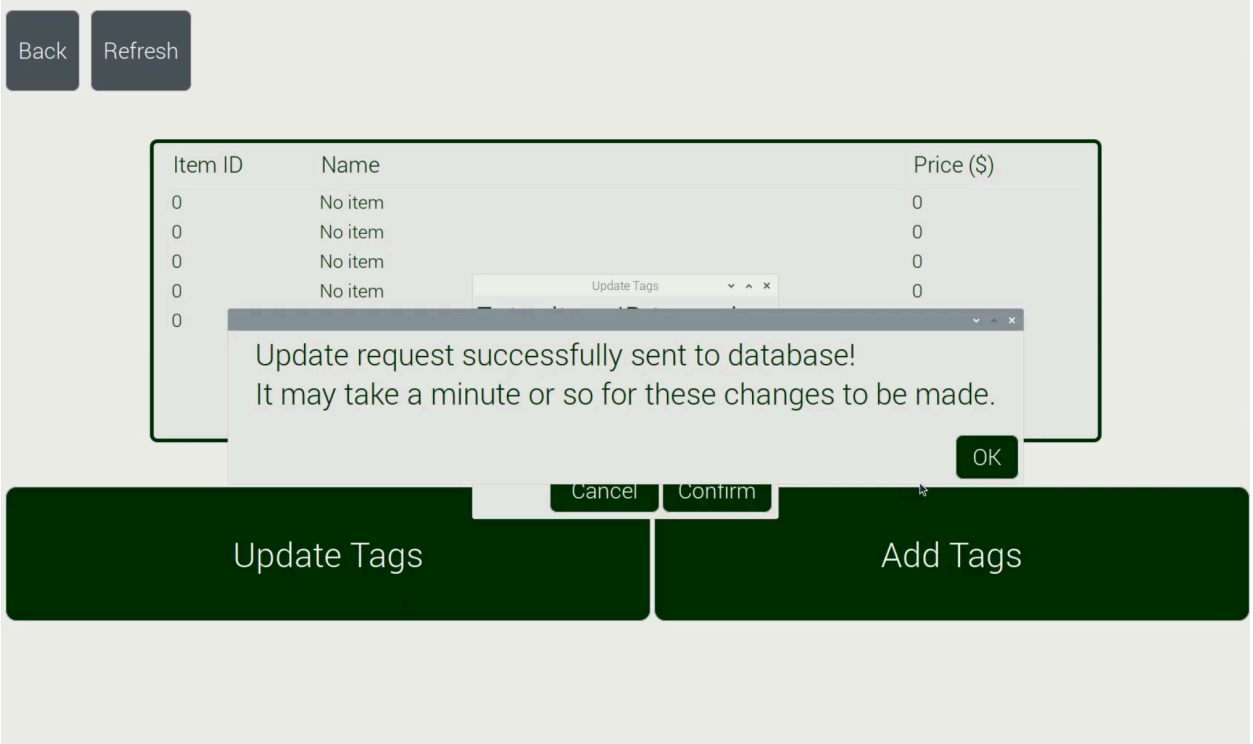
Once the tags have been added to the database. They will be assigned to an item with item ID 0, the ‘No item’ item. Here is what the user should expect to see after confirming the add tag function and clicking ‘Refresh’ to rescan the tags:



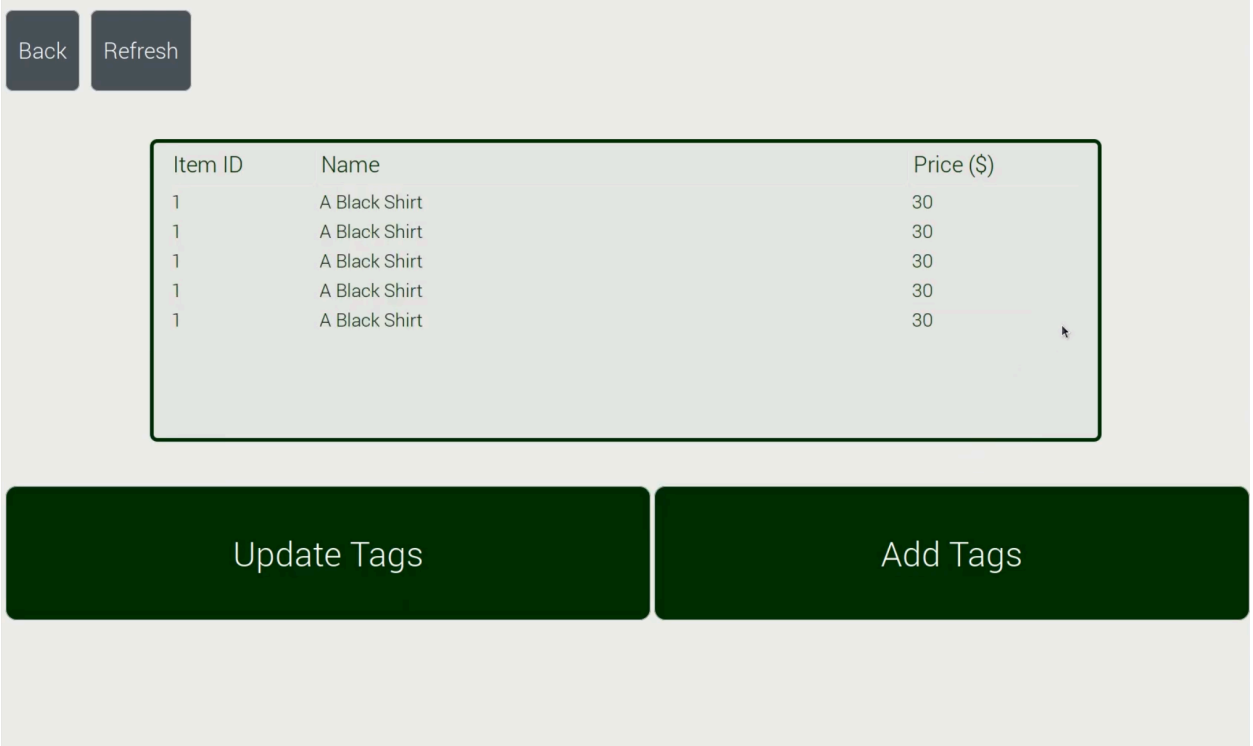
With these tags in the database, an admin may select the ‘Update Tags’ button to reprogram the tags to represent a certain item. This may also be done for tags already programmed to any item, allowing reprogramming of tags repeatedly. After selecting the ‘Update Tags’ button, a dialog box will open allowing the user to the input item ID of the item they wish to assign the tags in the control bin to. In this screenshot, the items are being assigned to the item with item ID ‘1’, in our case ‘A Black Shirt’:



Once the user confirms the tag update, a message will appear confirming the action:



After the tag update, the user may click the ‘Refresh’ button to initiate a rescan of the tags in the control bin and confirm that the database was updated correctly:



After the programming, the tags are ready to be deployed in the clothing store and purchased by customers, and reprogrammed again.

Critical Problems Solved

Hardware

Unlocking Mechanism

Initially, we were planning to use a system of 3D-printed gears powered by a DC motor to perform the unlocking mechanism. We decided to change this by using a linear actuator to perform the linear motion required to pull the rod to unlock the clothing pin.

Locking Mechanism

When testing the locking mechanism, a problem occurred where the linear actuator was not strong enough to pull the plunger out of the locked position due to the strength of the springs used and the friction between the lockButton and the plunger. Redesigns were made to use weaker springs, to stabilize the lockButton so that the plunger would not get stuck in the lockButton, and a smaller amount of movement from the plunger to reduce the friction between the plunger and the lockButton. We also tried to use some lubricant between the moving parts. With these changes, the linear actuator was able to pull the plunger out of the locked state.

3D-Printed Case and PCB Mounting and Location

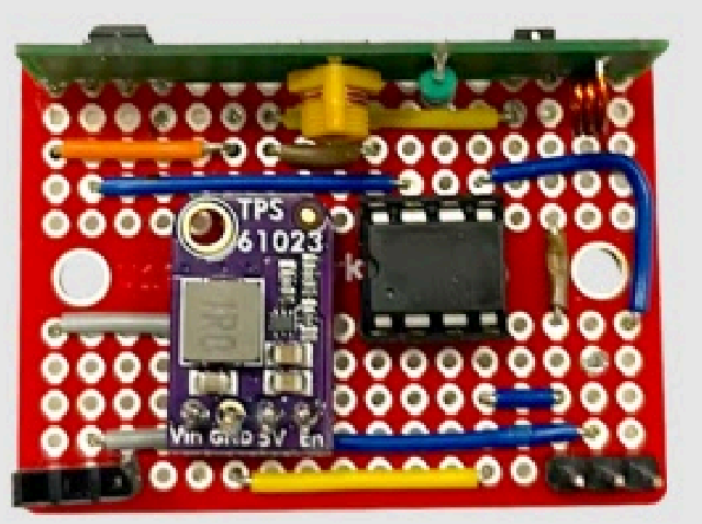
The PCB was going to be placed vertically along the bottom base but required a large space to account for the height of components. After a few PCB design and component changes, we were able to decrease the size of the overall PCB and change the location of the PCB to rest under the middle mounting plate. Upon decreasing the PCB, we were also able to decrease the overall size of the 3D-printed case.

Actuator Mounting Design

When creating the 3D-printed mounts for the actuator, we were able to design c-shaped mounting brackets to keep the actuator in place on each side. Since the gear on the actuator was sticking out of the actuator base, we noticed that the gear was interfering with the mounting brackets. To resolve this issue, we redesigned the mounting brackets to be thinner and created a gap where the gear would be. This gave the gear enough clearance to rotate while ensuring the actuator was mounted securely in the smart tag.

Receiving Actuator Signals on the Tag

Designing a system to receive RF data on the tag for controlling the actuator proved to be challenging. This is due to the constraints of space and power on the tag. This receiving circuit is to have low physical footprint, and consume minimal energy while listening for incoming signals. After researching potential solutions, we determined that the heart of the circuit would be the ATtiny85, a low-power CMOS 8-bit microcontroller with 8 kilobytes of in-system programmable flash and 6 GPIO pins. This small microcontroller fits the constraints of low power usage and small physical footprint. Here is the ATtiny85 within the final tag circuit design (it is the black PDIP case):



PCB Power Supply

When creating the PCB, the system was drawing more power than initially anticipated. A coin battery was not supplying enough current to support all three loads, thus, a new battery had to be selected to account for the needed current ratings. In addition, the IC did not have the proper current ratings given an input voltage of 3.7V. Thus, a new IC had to be selected to account for the proper current ratings required. This troubleshooting process caused significant stress as the original PCB was created using an external source from a Chinese manufacturer. However, the new board had to be hand-soldered due to time constraints.

Tag Sizing

When designing the entire tag, the dimensions of the tag depended on the parts expected to fit inside. As we encountered some issues with the PCB and redesigns of the unlocking mechanism, there were some changes that needed to be made to the sizing of the outer walls and inner mounting parts to ensure the fit of all parts inside the tag.

Software

Signal Processing on the ATtiny85

Processing incoming RF communications on the ATtiny85 was challenging due to the resource constraints of the device. The ATtiny85 does not have an operating system, so debugging activities required an LED to track the state of the program logic. In the end, ensuring the ATtiny85 was running on its 8MHz internal clock and setting the pulse length of the transmission to a relatively long 1 second to ensure that timing is not a problem when the Raspberry Pi is sending codes to the ATtiny85.

Integration of Graphical User Interface Elements and Backend Processes

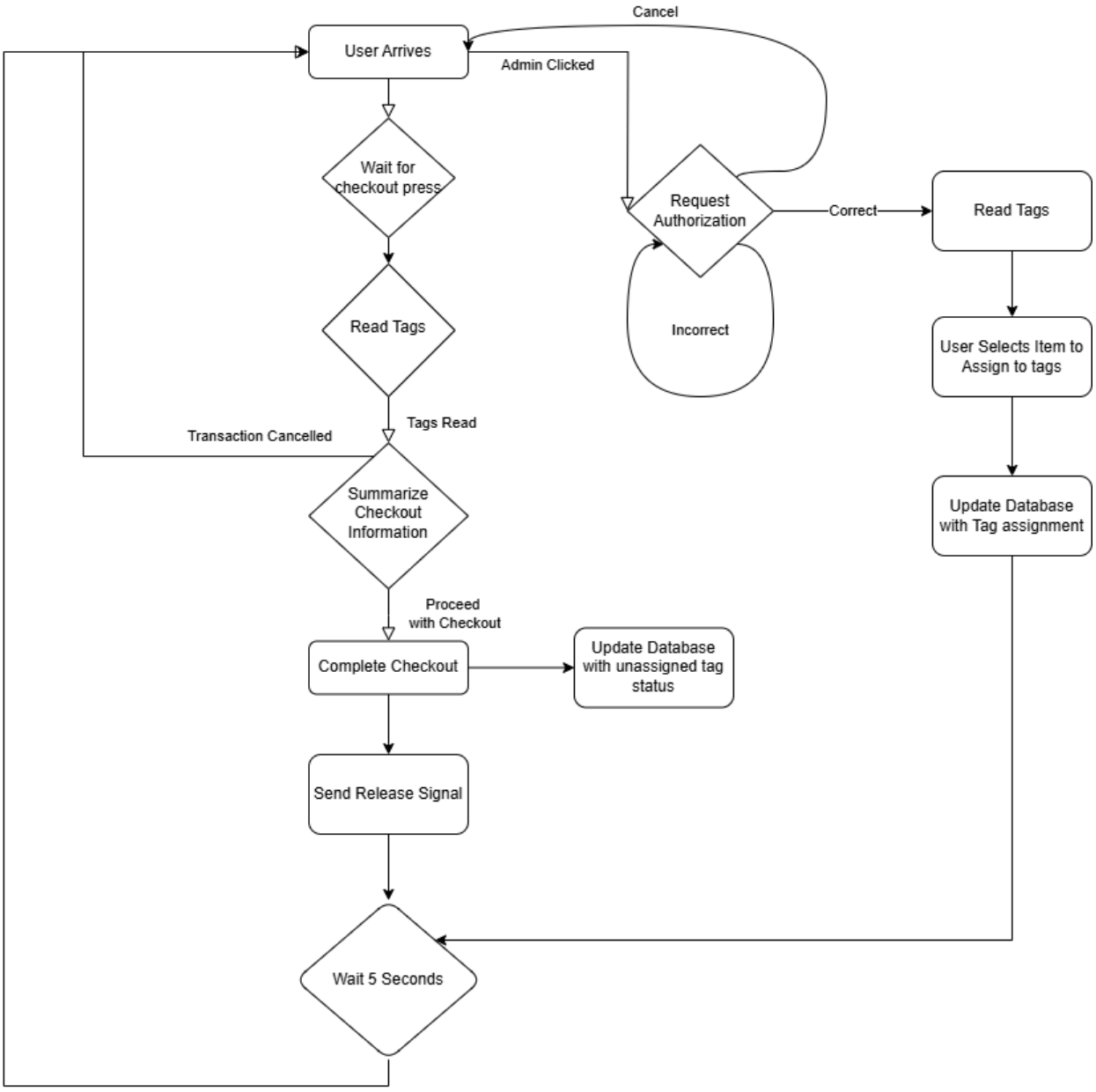
We initially planned to create our GUI using Python because of the language's rich front-end library resources. However, we noted that our back-end processes were all written in the C programming

language. This meant that we would need to create some interface for the Python GUI process to communicate with the C backend processes.

We considered a few options: (1) running all C processes through OS calls, (2) creating a network socket interface between the Python and C code bases, and (3) using GTK, a toolkit for creating GUIs compatible to be written in C. We first decided that OS calls would be too burdensome due to the number of C processes and the need for data to be shared. In the end, we decided to use GTK over a network socket interface due to the extra work needed to create that network socket interface.

System Flow for GUI

For the software end, and specifically in the UI implementation, we have decided to slightly change the flow of our system to better reflect potential user flows and to ease the experience of using the system. These changes were made to accommodate for the fact that many users may have previously learned habits from shopping in various different stores. We believe that instead of having the sensor constantly listening for RFID signals, the user to begin the transaction when they arrive at the bin, and then to proceed to be guided through the checkout process. We believe that this way there will be an easier adoption phase of our product and ease the user experience. Below is an outline of the newly formulated GUI system flow.


Cancel
User Arrives
Admin Clicked
Wait for checkout press
Request Authorization
Correct
Read Tags
Read Tags
Incorrect
User Selects Item to Assign to tags
Tags Read
Transaction Cancelled
Summarize Checkout Information
Update Database with Tag assignment
Proceed with Checkout
Complete Checkout
Update Database with unassigned tag status
Send Release Signal
Wait 5 Seconds

Conclusion

Over these eight months, our team successfully designed and implemented a fully functional Reusable Smart Clothing Tag System aimed at improving retail efficiency and reducing environmental waste. The final deliverables include three core components: the Smart Tag, the Control Bin, and the Backend System—each integrated to form a retail self-checkout solution.

The Smart Tag was engineered as a compact, reusable device that attaches to garments and features a spring-loaded locking mechanism controlled via a linear actuator. It includes an RFID tag for identification and a PCB powered by a rechargeable battery system with a boost converter. This tag automatically releases upon successful checkout, reducing the need for manual tag removal and improving customer experience.

The Control Bin acts as both a customer-facing checkout system and an administrative programming interface. It includes a Raspberry Pi 4, RFID scanner, RF transmitter, and user interface peripherals. Functionality includes real-time tag reading, database communication, and triggering the release mechanism within the smart tags.

The Backend System was developed using a SQLite3 database that maps RFID tag IDs to corresponding clothing item data. A command-line interface and a GUI prototype were implemented to manage product-tag associations, perform checkouts, and facilitate administrative tasks.

Despite facing technical challenges—such as power supply limitations, actuator interference, and UI integration—we resolved each through testing, hardware redesigns, power system updates, and the transition from Python to GTK3 for GUI development in C. Collectively, the system demonstrates our system combines the use of software, hardware, and engineering design skills to create a product with real-world development.

Future Plan

The potential market for TagSense is a focus on the commercial retail market, introducing a reusable automated technology instead of a simple barcode checkout system. This technology will increase automation processes and sustainability measures. Apparel and sports clothing businesses can introduce the self-checkout system to enhance customer experience, improve customer satisfaction, reduce line abandonment and bolster peak retail seasons. The system is designed with a specific market value and can be expanded to every fast fashion retail store in the industry. Another specific use could be for outlet stores, where price changes can be completed in the backend and during major sales or the holiday season can reduce congestion and dissatisfaction with long wait times.

Refining Tag Design

During the smart tag build, we encountered various issues where we had to replace components and redesign the prototype. Given the working product, we can refine the tag design by reducing the size and

empty space. This can be accomplished by choosing a smaller battery and making the custom PCB smaller, making more use of the spacing within the tag.

Charging Capabilities

Our current prototype includes a rechargeable battery, but requires disassembling the smart tag to retrieve the battery to charge. For future steps, we would want to implement wireless charging for the smart tags to provide an efficient method to prolong the usage of the smart tags. When smart tags are placed into a charging bin, they would be capable of automatically charging multiple tags at once.

User Interaction

The control bin is the only source where the smart tags can be read with the description and price of clothing items displayed. We would like to develop a phone application where customers can download our app and scan smart tags from their phone's built-in RFID reader. Furthermore, the application would have shop inventory information stored in the database for customers to view. This would enhance their overall shopping experience.

Security Capabilities

The clothing tag can be expanded to account for theft protection. As the system utilizes an RFID system, security RFID readers can be placed at the store's entrances to alert staff and those in the area of theft. The system can also detect for unpaid items as the store tags can only be removed using administrative uses and instore purchases.

Locking mechanism

The current prototype uses a pin to lock onto a piece of clothing but there were thoughts of changing the pin to a different mechanism to avoid damage to the clothing when the tag is secured. The locking mechanism that was used was also not the most reliable as it was susceptible to malicious attacks and will need to be revised and thoroughly tested before the tag will be able to be used as a security feature.

Detailed Individual Self-Reflection and Assessment

Brian Ho

Satisfactory. 20%

Overall, tasks were completed promptly. I heavily focused on the electrical design and creation portion of the project, spearheading the electrical portions of the smart tag system. Additionally, I assisted the team in areas that required more effort.

Designing Boost Converter/ Battery/ Electrical Portability: 95%
Using LTspice, drafted and arranged a boost converter to increase voltage from a 3V coin battery to the needed 5V output. However, the coin battery did not yield sufficient current, so a design change was required. Sourced and designed a new boost converter and battery system to fulfill the portability of the smart tag.

PCB Creation and Testing: 85%
The original custom PCB was created on EasyEDA and manufactured in China. After some testing, however, due to a design defect, the completed project was delayed and not finished until later. A significant amount of time was spent testing and troubleshooting the circuitry of device loads. Once the issue was identified and a solution was determined, a new PCB was designed and soldered.

UI Design and Implementation: 20%
Designed the original shell of UI and helped make small frontend changes.

Capstone Video: 30%
Storyboarded, filmed and acted for the Capstone video.

Capstone Poster: 50%
Designed the expo day poster, creating diagrams and adding descriptions to explain the journey and process for the project.

Justin Covach

Contributions Overall 20%

RFID System Implementation (75%):

Completed the implementation on schedule and verified its performance. This was done using the MercuryAPI docs and includes the RFID_utils.c.c and part of the SmartTagSystem_CLI.c files. Created all the util.c files and all the base commands for the system loop. Also integrated previously implemented signalling program into the main loop.

Database Connection (90%):

Implemented the SQLite3 database as well as populating it with different values. This was implemented in the DB_utils.c.c file. These functions were integrated into the SmartTagSystem_CLI.c program. Implemented and helped verify functionality of these functions as well as helped implement them into the final UI design.

CLI Implementation (100%):

Completed the CLI implementation as a preliminary to the UI system to verify the flow of the system. This implements all of the previous code into one system called SmartTagSystem_CLI.c. Also configured the libraries and MakeFile for this project.

UI implementation (30%):

I did a lot of the UI styling in the CSS and also helped build some components in the pages such as the checkout table. Helped debug the different issues and design problems as we progressed through the design.

Circuit Debugging (5%):

I helped to debug our PCB circuit that was having issues working consistently. I was able to determine the root of one of our problems which was a loose jumper connection to one of our main circuit components.

Capstone Video (5%):

I handled the recording with the camera, set up shots and various different ideas as well as contributed to some acting on the day of the recording.

Overall:

I think overall I contributed well and provided a good final product fot my personal deliverables. I think I completed them on time and according to our schedule to maintain good progress. I think I worked well with the team to manage work and collaborate on the required areas. Overall I think that I was flexible to work with and provided good value to the team throughou the whol project. I think it was an excellent performance overall with some minor sections that may have needed improvement throughout the process but overall still an excellent

Ben Miles

Satisfactory. Contributed to 20% of the overall project.

Overall, I rate my performance as satisfactory. My deliverables were almost always on time, and I was available to communicate effectively in our team meetings and group chats. For over half the project, I was mainly focused on the actuator signaling circuit and logic design and was able to get this ready and working well before Expo day. I was then able to support testing with other teammates, and help code a substantial amount of the GUI, mainly focusing on GTK3 functionality. The items below list my contributions in detail:

PCB creation and testing (10%)
I helped design the PCB by contributing a small section around the ATtiny85 for receiving signals from the control bin's Raspberry Pi 4 to decide whether it should open the tag using the linear actuator. This involved selecting a 434.17 MHz transmitter and receiver pair, and creating simple circuits around the Raspberry Pi 4 (to transmit) and the ATtiny85 (to receive). I also helped testing the end to end functionality of the PCB's performance from the receiving of the Raspberry Pi's actuating signal to the control of the actuator.

Actuator signaling logic (90%)
I was responsible for writing the code for the Raspberry Pi 4 to send signals to the ATtiny85, and the code for the ATtiny85 to receive the signals and control the linear actuator. This involved testing and creating a circuit to flash the ATtiny85 from the Raspberry Pi using the Arduino IDE and the avrdude library. I also selected an RF communication library compatible with both the Raspberry Pi and the ATtiny85, the rc-switch library, and wrote the Pi's transmit code in C++ and the ATtiny85's code in the Arduino '.ino' style. To control the linear actuator, I selected and configured the SoftwareServo library for the ATtiny85. In good time for the end of our project, we had a working actuator signaling circuit pair and logic in which the ATtiny85 could listen for a unique 16-bit open code sent from the Raspberry Pi 4 in the control bin then move the actuator in a predefined way.

UI implementation (30%)
I helped write code in C using GTK3 to create our GUI. My main contribution was completing the functionality of the GUI rather than the design or the aesthetics. As no one in our team had used GTK3 before, this involved researching how GTK3 works, and figuring out how to implement functionalities such as linking functions to button clicks and having multiple accessible pages.

Capstone video (5%)
I helped with filming some of the shots for our Capstone day videos. Namely, I helped setup for filming the project hardware closeups and live project demo shots. I also screen recorded shots of our code, GUI, and database. I also provided a voice-over of the environmental benefits and tagline for our main Capstone video.

Danielle Fong

Satisfactory - 20% contribution of the overall project

Deliverables were mostly completed on time, but there were many design changes that had to be made on the hardware side. Due to these design changes and wait times for 3D printing and printing availability, there were slight delays when assembling the printed model. Furthermore, there were many adjustments to ensure the stability of the locking mechanism and the ability of the actuator to counter spring and friction forces, creating a seamless pin release. Lastly, the video and poster was completed in the last few weeks to ensure everything required was presented at the expo.

Contributions

Actuator mechanism and design for unlocking pin: 40%

I helped with the design of how the actuator would operate to unlock the clothing pin in the system. This involved testing the strength of the actuator against the spring force in the rod and ensuring the actuator was mounted in line with the locking block. I also designed the mounting brackets to ensure that no interference occurs for the linear actuator when in motion, since we previously had issues with that when testing.

Smart tag casing and component arrangements: 50%

I worked on the layout of components enclosed in the smart tag. This included attempting to decrease the overall size of the tag where possible, but ensuring there was still enough clearance for each component to get the optimal size. Due to design and component changes along the way, there were many iterations that were performed for this task.

Hardware build and testing: 25%

I contributed to the overall hardware build and testing. This includes assembling the 3D-printed parts, testing the locking/unlocking mechanism, finding where things could be improved or changes needed to be made, and updating the models accordingly for reprints and rebuilds.

UI Implementation: 20%

I worked on the UI design for the checkout system.

Capstone Video: 70%

I worked on creating and editing the final capstone video.

Capstone Poster: 50%

I designed the expo day poster, creating diagrams and adding descriptions to explain the overall project.

Isaac Kong

Satisfactory 20%

Delivery of 3D printed parts was slow. Many design changes had to be made to the model as 3D printing was underway. Due to this, many of the 3D parts needed to be reprinted. There was very limited 3D printing booking from the library and so 3D printing needed to be sourced elsewhere to keep up with the number of prints required for the designing process.

Contributions

Actuator mechanism and design for unlocking pin: 60%

I worked on the design of the locking mechanism. This includes the initial process of figuring out what would be needed of the system, 2D sketches of the proposed mechanism, 3D Modeling of the mechanism, testing of the physical parts, and debugging the design after testing.

Smart tag casing and component arrangements: 50%

I worked on the layout of the parts in the tag. This includes setting the initial dimensions of the tag, accommodating the design for changes to the expected PCB, revisions to the design to reduce the size of the tag, and planning the location of the screw holes for mounting to ensure a stable product.

Hardware build and testing: 75%

I worked on the manufacturing, assembly, and testing of the parts. This includes preparing 3D models for printing, fine tuning of parts after assembly, adjusting the models after testing, and communication with the person helping us print the parts.

References

[1] United States consumer & retail report - Q4 2024. (2024). (). London: Fitch Solutions Group Limited. Retrieved from ProQuest One Business Retrieved from http://libaccess.mcmaster.ca/login?url=https://www.proquest.com/reports/united-states-consumer-amp-retail-report-q4-2024/docview/3103755401/se-2

[2] Canada consumer & retail report - Q1 2024. (2024). (). London: Fitch Solutions Group Limited. Retrieved from ProQuest One Business Retrieved from http://libaccess.mcmaster.ca/login?url=https://www.proquest.com/reports/canada-consumer-amp-retail-report-q1-2024/docview/2896060336/se-2

[3] Khodke, K. (2021, February 20). Don't tag along: Reimagining clothing tags and care labels for a circular economy. Medium. https://kalyanikhodke.medium.com/dont-tag-along-reimagining-clothing-tags-and-care-labels-for-a-circular-economy-a24102264d92#:~:text=The%20tags%2C%20put%20together%2C%20equal,landfills%20or%20to%20our%20oceans